

Lectures with examples

Fixed Point Representation

- Fixed point is the simplest way to represent real numbers digitally
 - Also known as **Qm.n format**, where:
 - **m** bits for the integer part
 - **n** bits for the fractional part
- **Key Properties:**
 - Range: $[-(2^m), 2^m - 2^{-n}]$
 - Resolution: 2^{-n} (smallest representable difference)
 - Storage: $m + n + 1$ bits total (including sign bit)
- **Limitations:**
 - Fixed range of representable numbers
 - Trade-off between range (m) and precision (n)
 - Cannot efficiently handle very large or very small numbers

Floating point

The numbers in computer memory are typically represented as **floating point numbers**.

A floating point number is represented as:

$$\text{number} = \text{significand} \times \text{base}^{\text{exponent}}$$

where:

- *significand* is an integer
- *base* is a positive integer
- *exponent* is an integer (can be negative)

For example:

$$1.2 = 12 \cdot 10^{-1}$$

Floating point: formula

$$f = (-1)^s 2^{(p-b)} \left(1 + \frac{d_1}{2} + \frac{d_2}{2^2} + \dots + \frac{d_m}{2^m} \right),$$

where $s \in \{0, 1\}$ is the sign bit, $d_i \in \{0, 1\}$ is the m -bit mantissa, $p \in \mathbb{Z}; 0 \leq p \leq 2^e$, e is the e -bit exponent, commonly defined as $2^e - 1$.

Can be thought as a uniform m -bit grid between two sequential powers of 2.

Simple examples

There are many ways to write a number in scientific notation, but there is always a unique normalized representation, with exactly one non-zero digit to the left of the decimal point.

For example:

- $0.232 \times 10^3 = 23.2 \times 10^1 = 2.32 \times 10^2 = \dots$
- $01001 = 1.001 \times 2^3 = \dots$

Example 1: What's the normalized representation of 00101101.101?

$$0.0001101001110 = 1.110100111 \times 2^{-4}$$

Task: Prove that signed binary fixed-point numbers lay in the range $\pm(2^m - 2^{-n})$, where m, n are sizes of an integer and fractional part, respectively.

Solution: The maximal value in a binary notation is zero sign bit and all ones in the main parts. That gives

$$\sum_{i=0}^{m-1} 2^i = \{\text{geometric sum for finite terms}\} = \frac{1 \cdot (2^{(m-1)+1} - 1)}{(2 - 1)} = 2^m - 1$$

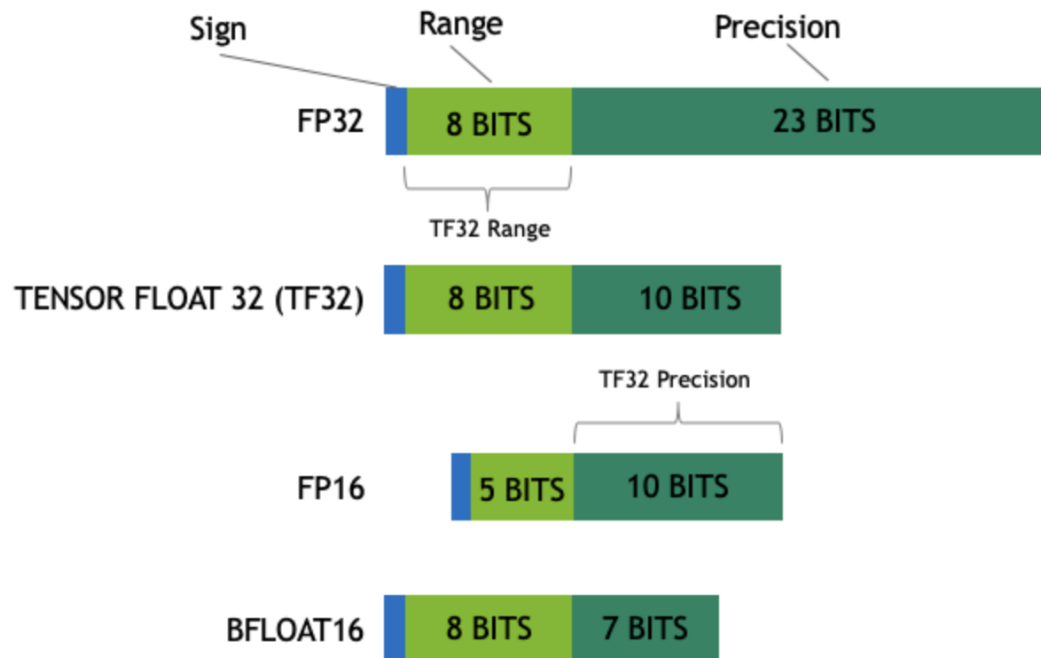
for the integer part and

$$\sum_{i=1}^n 2^{-i} = \frac{2^{-1} \cdot (2^{-n} - 1)}{(2^{-1} - 1)} = 1 - 2^{-n}$$

for the fractional.

The maximal value in total is equal to $(2^m - 1) + (1 - 2^{-n}) = 2^m - 2^{-n}$.

For the minimal value, we should simply take the sign bit as one, so the value is the maximal value but negative: $-(2^m - 2^{-n})$.



Kahan summation

The following algorithm gives $2\epsilon + \mathcal{O}(n\epsilon^2)$ error, where ϵ is the machine precision.

- The reason of the loss of significance in summation is operating with numbers of different magnitude
- The main idea of Kahan summation is to keep track of small errors and aggregate them in a separate variable
- This approach is called *compensated summation*

```
s = 0
c = 0
for i in range(len(x)):
    y = x[i] - c
    t = s + y
    c = (t - s) - y
    s = t
```

Distances and norms

- Norm is a **qualitative measure of smallness of a vector** and is typically denoted as $\|\mathbf{x}\|$.

The norm should satisfy certain properties:

- $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|$
- $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality)
- If $\|\mathbf{x}\| = 0$ then $\mathbf{x} = \mathbf{0}$

The distance between two vectors is then defined as

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|.$$

Standard norms

Euclidean norm:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2},$$

p-norm

Euclidean norm, or 2-norm, is a subclass of an important class of *p*-norms:

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

There are two very important special cases:

- **Infinity norm, or Chebyshev norm** is defined as the element of the maximal absolute value:

$$\|\mathbf{x}\|_\infty = \max_i |x_i|$$

- **L_1 norm (or Manhattan distance)** which is defined as the sum of modules of the elements of $\mathbf{x}$:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$

Equivalence of the norms

All norms are equivalent in the sense that

$$C_1 \|\mathbf{x}\|_* \leq \|\mathbf{x}\|_{**} \leq C_2 \|\mathbf{x}\|_*$$

for some positive constants $C_1(n), C_2(n)$, $\mathbf{x} \in \mathbb{R}^n$ for any pairs of norms $\|\cdot\|_*$ and $\|\cdot\|_{**}$. The equivalence of the norms basically means that if the vector is small in one norm, it is small in another norm. However, the constants can be large.

Proof Sketch (For Finite-Dimensional Spaces)

The proof of this theorem is foundational in functional analysis and is usually done by demonstrating that every norm on a finite-dimensional vector space (like $\mathbb{R}^n$) is equivalent to the **Euclidean norm** ($\|\cdot\|_2$).

Key Steps:

- 1. Define a Basis:** For $\mathbf{x} \in \mathbb{R}^n$, let $\mathbf{x} = \sum_{i=1}^n x_i \mathbf{e}_i$, where $\{\mathbf{e}_i\}$ is the standard basis.
- 2. Show Boundedness (C_2 part):**
 - Show that $\|\mathbf{x}\| \leq K \cdot \|\mathbf{x}\|_\infty$ for some constant K .
 - This is achieved using the triangle inequality: $\|\mathbf{x}\| = \|\sum x_i \mathbf{e}_i\| \leq \sum |x_i| \|\mathbf{e}_i\|$.
 - Since $|x_i| \leq \|\mathbf{x}\|_\infty$, it can be shown that $\|\mathbf{x}\| \leq (\sum \|\mathbf{e}_i\|) \|\mathbf{x}\|_\infty$, where $C_2 = \sum \|\mathbf{e}_i\|$.
- 3. Show Lower Bound (C_1 part):**
 - Show that $\|\mathbf{x}\|_\infty \leq C \cdot \|\mathbf{x}\|$. This is the harder part and typically involves a **compactness argument** using the Heine-Borel theorem.
 - The unit sphere $S = \{\mathbf{x} \in \mathbb{R}^n \mid \|\mathbf{x}\|_\infty = 1\}$ is compact.
 - The function $f(\mathbf{x}) = \|\mathbf{x}\|$ is continuous on $\mathbb{R}^n$ and thus on S .
 - Since $f(\mathbf{x})$ is continuous on a compact set, it attains a minimum $m = \min_{\mathbf{x} \in S} f(\mathbf{x})$. Since S does not contain the zero vector, $m > 0$.
 - Using the homogeneity property, it can be proven that $\|\mathbf{x}\| \geq m \cdot \|\mathbf{x}\|_\infty$.
 - By setting $C_1 = 1/m$, we get $\|\mathbf{x}\|_\infty \leq \frac{1}{m} \|\mathbf{x}\|$.
- 4. Conclusion:** By transitivity, since any arbitrary norm $\|\cdot\|$ is equivalent to the infinity norm ($\|\cdot\|_\infty$), all norms are equivalent to each other.

What is a stable algorithm?

And we finalize the lecture by the concept of **stability**.

- Let $\mathbf{x}$ be an object (for example, a vector)
- Let $f(\mathbf{x})$ be the function (functional) you want to evaluate

You also have a **numerical algorithm** `alg(x)` that actually computes **approximation** to $f(\mathbf{x})$.

The algorithm is called **forward stable**, if

$$\|\text{alg}(\mathbf{x}) - f(\mathbf{x})\| \leq \epsilon$$

The algorithm is called **backward stable**, if for any $\mathbf{x}$ there is a close vector $\mathbf{x} + \delta\mathbf{x}$ such that

$$\text{alg}(\mathbf{x}) = f(\mathbf{x} + \delta\mathbf{x})$$

and $\|\delta\mathbf{x}\|$ is small.

Matrices and norms

- Recall vector norms that allow to evaluate distance between two vectors or how large are elements of a vector.
- How to generalize this concept to matrices?
- A trivial answer is that there is no big difference between matrices and vectors, and here comes the simplest matrix norm — **Frobenius norm**:

$$\|\mathbf{A}\|_{\text{F}} \stackrel{\text{def}}{=} \left(\sum_{i=1}^n \sum_{j=1}^m |a_{ij}|^2 \right)^{1/2}$$

Matrix norms

$\|\cdot\|$ is called a **matrix norm** if it is a vector norm on the vector space of $n \times m$ matrices:

1. $\|\mathbf{A}\| \geq 0$ and if $\|\mathbf{A}\| = 0$ then $\mathbf{A} = \mathbf{O}$
2. $\|\alpha\mathbf{A}\| = |\alpha|\|\mathbf{A}\|$
3. $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$ (triangle inequality)

Additionally some norms can satisfy the **submultiplicative property**

- $$\|\mathbf{AB}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$$
- These norms are called **submultiplicative norms**.
- The submultiplicative property is needed in many places, for example in the estimates for the error of solution of linear systems (we will cover this topic later).
- Example of a non-submultiplicative norm is **Chebyshev norm**

$$\|\mathbf{A}\|_{\text{C}} = \max_{i,j} |a_{ij}|$$

Operator norms

- The most important class of the matrix norms is the class of **operator norms**. They are defined as

$$\|\mathbf{A}\|_{*,**} = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_*}{\|\mathbf{x}\|_{**}}$$

where $\|\cdot\|_*$ and $\|\cdot\|_{**}$ are **vector norms**.

- It is easy to check that operator norms are submultiplicative if $\|\cdot\|_* = \|\cdot\|_{**}$. Otherwise, it can be non-submultiplicative, think about example.
- **Frobenius norm** is a matrix norm, but not an operator norm, i.e. you can not find $\|\cdot\|_*$ and $\|\cdot\|_{**}$ that induce it.
- This is a nontrivial fact and the general criterion for matrix norm to be an operator norm can be found in [Theorem 6 and Corollary 4](#). For $\|\cdot\|_* = \|\cdot\|_{**}$ let us check on the blackboard!

Matrix p -norms

Important case of operator norms are matrix p -norms, which are defined for $\|\cdot\|_* = \|\cdot\|_{**} = \|\cdot\|_p$.

Among all p -norms three norms are the most common ones:

- $p = 1$ (**Column-sum norm**):

$$\|\mathbf{A}\|_1 = \max_j \sum_{i=1}^n |a_{ij}|$$

- $p = 2$ (**Spectral norm**):

- Spectral norm, denoted by $\|\mathbf{A}\|_2$.
- *Definition:* $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^T \mathbf{A})}$, where $\lambda_{\max}$ is the largest eigenvalue.

- $p = \infty$ (**Row-sum norm**):

$$\|\mathbf{A}\|_\infty = \max_i \sum_{j=1}^m |a_{ij}|$$

Proof Sketch for $p = \infty$ (Row-Sum Norm)

The goal is to show $\|\mathbf{A}\|_\infty = \max_i \sum_j |a_{ij}|$, where the induced operator norm is:

$$\|\mathbf{A}\|_{\infty} = \sup_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_{\infty}}{\|\mathbf{x}\|_{\infty}}$$

1. Upper Bound ($\leq$):

- By definition of the infinity vector norm, $\|\mathbf{Ax}\|_{\infty} = \max_i |(\mathbf{Ax})_i|$.
- $|(\mathbf{Ax})_i| = |\sum_j a_{ij}x_j| \leq \sum_j |a_{ij}||x_j| \leq \left(\sum_j |a_{ij}|\right) \max_k |x_k| = \left(\sum_j |a_{ij}|\right) \|\mathbf{x}\|_{\infty}$.
- Therefore, $\|\mathbf{Ax}\|_{\infty} \leq \left(\max_i \sum_j |a_{ij}|\right) \|\mathbf{x}\|_{\infty}$.
- Dividing by $\|\mathbf{x}\|_{\infty}$ gives the upper bound: $\|\mathbf{A}\|_{\infty} \leq \max_i \sum_j |a_{ij}|$.

2. Lower Bound ($\geq$):

- Let i_0 be the row that achieves the maximum row sum:
 $\sum_j |a_{i_0j}| = \max_i \sum_j |a_{ij}|$.
- Construct a test vector $\mathbf{x}$ such that its elements are $x_j = \text{sgn}(a_{i_0j})$, where $\text{sgn}(0) = 1$. This ensures that $|x_j| \leq 1$ for all j , so $\|\mathbf{x}\|_{\infty} = 1$.
- For this specific $\mathbf{x}$, the i_0 -th component of $\mathbf{Ax}$ is
 $(\mathbf{Ax})_{i_0} = \sum_j a_{i_0j}x_j = \sum_j a_{i_0j} \cdot \text{sgn}(a_{i_0j}) = \sum_j |a_{i_0j}|$.
- Since $\|\mathbf{Ax}\|_{\infty} = \max_k |(\mathbf{Ax})_k| \geq |(\mathbf{Ax})_{i_0}|$, we have
 $\|\mathbf{Ax}\|_{\infty} \geq \sum_j |a_{i_0j}| = \max_i \sum_j |a_{ij}|$.
- Since $\|\mathbf{x}\|_{\infty} = 1$, the quotient is $\frac{\|\mathbf{Ax}\|_{\infty}}{\|\mathbf{x}\|_{\infty}} \geq \max_i \sum_j |a_{ij}|$, which implies the lower bound.

Since we proved $\leq$ and $\geq$, the two sides must be equal.

Easy Examples for Matrix p -norms

Let the matrix $\mathbf{A}$ be:

$$\mathbf{A} = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$$

1. $\|\mathbf{A}\|_1$ (Column-sum norm)

The norm is the maximum absolute column sum:

- Column 1 sum: $|1| + |3| = 4$
- Column 2 sum: $|-2| + |4| = 6$

$$\|\mathbf{A}\|_1 = \max(4, 6) = 6$$

2. $\|\mathbf{A}\|_2$ (Spectral norm)

The norm is $\sqrt{\lambda_{\max}(\mathbf{A}^T \mathbf{A})}$.

- $\mathbf{A}^T \mathbf{A} = \begin{pmatrix} 1 & 3 \\ -2 & 4 \end{pmatrix} \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 10 & 10 \\ 10 & 20 \end{pmatrix}$
- Eigenvalues (λ): $\det(\mathbf{A}^T \mathbf{A} - \lambda \mathbf{I}) = (10 - \lambda)(20 - \lambda) - 100 = 0$.
- $\lambda^2 - 30\lambda + 100 = 0$.
- $\lambda_{\max} \approx 26.18$.

$$\|\mathbf{A}\|_2 = \sqrt{26.18} \approx 5.1166$$

3. $\|\mathbf{A}\|_\infty$ (Row-sum norm)

The norm is the maximum absolute row sum:

- Row 1 sum: $|1| + |-2| = 3$
- Row 2 sum: $|3| + |4| = 7$

$$\|\mathbf{A}\|_\infty = \max(3, 7) = 7$$

Scalar product

While norm is a measure of distance, the **scalar product** takes angle into account.

It is defined as

- **For vectors:**

$$(\mathbf{x}, \mathbf{y}) = \mathbf{x}^* \mathbf{y} = \sum_{i=1}^n \bar{x}_i y_i,$$

where $\bar{x}$ denotes the **complex conjugate** of x . The Euclidean norm is then

$$\|\mathbf{x}\|_2 = \sqrt{(\mathbf{x}, \mathbf{x})},$$

or it is said the norm is **induced** by the scalar product.

- **For matrices (Frobenius scalar product):**

$$(\mathbf{A}, \mathbf{B})_F = \sum_{i=1}^n \sum_{j=1}^m \bar{a}_{ij} b_{ij} \equiv \text{trace}(\mathbf{A}^* \mathbf{B}),$$

where $\text{trace}(\mathbf{A})$ denotes the sum of diagonal elements of $\mathbf{A}$. One can check that $\|\mathbf{A}\|_F = \sqrt{\text{trace}(\mathbf{A}\mathbf{A}^T)}$.

Task: Compute a Frobenius norm of

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

Solution:

The Frobenius norm $\|\mathbf{A}\|_F$ is defined as $\sqrt{\sum_{i=1}^n \sum_{j=1}^m |a_{ij}|^2}$.

Calculation of the sum of squared elements:

$$1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

Therefore, the Frobenius norm is:

$$\|\mathbf{A}\|_F = \sqrt{30}$$

Remark. The angle between two vectors is defined as

$$\cos \phi = \frac{(\mathbf{x}, \mathbf{y})}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}.$$

Similar expression holds for matrices.

- An important property of the scalar product is the **Cauchy-Schwarz-Bunyakovski inequality**:

$$|(\mathbf{x}, \mathbf{y})| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$$

Easy Example for Vector Scalar Product

Let $\mathbf{x}$ and $\mathbf{y}$ be real vectors:

$$\mathbf{x} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} -1 \\ 0 \\ 4 \end{pmatrix}$$

Since the vectors are real, the complex conjugate $\bar{x}_i$ is just x_i , and $\mathbf{x}^* = \mathbf{x}^T$.

The scalar product $(\mathbf{x}, \mathbf{y})$ is:

$$(\mathbf{x}, \mathbf{y}) = (1)(-1) + (2)(0) + (3)(4)$$

$$(\mathbf{x}, \mathbf{y}) = -1 + 0 + 12 = 11$$

The angle ϕ between them would be determined by:

$$\cos \phi = \frac{11}{\sqrt{(1^2 + 2^2 + 3^2)}\sqrt{(-1^2 + 0^2 + 4^2)}} = \frac{11}{\sqrt{14}\sqrt{17}} \approx 0.709$$

Matrices preserving the norm

- For **stability** it is really important that the error does not grow after we apply some transformations.
- Suppose you are given $\hat{\mathbf{x}}$ — the approximation of $\mathbf{x}$ such that,

$$\frac{\|\mathbf{x} - \hat{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \epsilon.$$

- Let us calculate a linear transformation of $\mathbf{x}$ and $\hat{\mathbf{x}}$:

$$\mathbf{y} = \mathbf{U}\mathbf{x}, \quad \hat{\mathbf{y}} = \mathbf{U}\hat{\mathbf{x}}.$$

- When building new algorithms, we want to use transformations that do not increase (or even preserve) the error:

$$\frac{\|\mathbf{y} - \hat{\mathbf{y}}\|}{\|\mathbf{y}\|} = \frac{\|\mathbf{U}(\mathbf{x} - \hat{\mathbf{x}})\|}{\|\mathbf{U}\mathbf{x}\|} \leq \epsilon.$$

- The question is for which kind of matrices the norm of the vector **will not change**, so that

$$\frac{\|\mathbf{U}(\mathbf{x} - \hat{\mathbf{x}})\|}{\|\mathbf{U}\mathbf{x}\|} = \frac{\|\mathbf{x} - \hat{\mathbf{x}}\|}{\|\mathbf{x}\|}.$$

- For the euclidean norm $\|\cdot\|_2$ the answer is **unitary (or orthogonal in the real case)** matrices.

Unitary (orthogonal) matrices

- Let $\mathbf{U}$ be complex $n \times n$ matrix, and $\|\mathbf{U}\mathbf{z}\|_2 = \|\mathbf{z}\|_2$ for all $\mathbf{z}$.
- This can happen **if and only if** (can be abbreviated as iff)

$$\mathbf{U}^* \mathbf{U} = \mathbf{I}_n,$$

where $\mathbf{I}_n$ is an identity matrix $n \times n$.

- Complex $n \times n$ square matrix is called **unitary** if

$$\mathbf{U}^* \mathbf{U} = \mathbf{U} \mathbf{U}^* = \mathbf{I}_n,$$

which means that columns and rows of unitary matrices both form orthonormal basis in $\mathbb{C}^n$.

- For rectangular matrices of size $m \times n$ ($n \neq m$) only one of the equalities can hold
 - $\mathbf{U}^* \mathbf{U} = \mathbf{I}_n$ — left unitary for $m > n$
 - $\mathbf{U} \mathbf{U}^* = \mathbf{I}_m$ — right unitary for $m < n$
- In the case of real matrices $\mathbf{U}^* = \mathbf{U}^T$ and matrices such that

$$\mathbf{U}^T \mathbf{U} = \mathbf{U} \mathbf{U}^T = \mathbf{I}$$

are called **orthogonal**.

Unitary matrices

Important property: a product of two unitary matrices is a **unitary matrix**:

$$(\mathbf{U}\mathbf{V})^* \mathbf{U}\mathbf{V} = \mathbf{V}^* (\mathbf{U}^* \mathbf{U}) \mathbf{V} = \mathbf{V}^* \mathbf{I} \mathbf{V} = \mathbf{V}^* \mathbf{V} = \mathbf{I},$$

- Later we will show that there are types of matrices (**Householder reflections** and **Givens rotations**) composition of which is able to produce arbitrary unitary matrix
- This idea is a core of some algorithms, e.g. **QR decomposition**

Unitary invariance of $\|\cdot\|_2$ and $\|\cdot\|_F$ norms

- For vector 2-norm we have already seen that $\|\mathbf{U}\mathbf{z}\|_2 = \|\mathbf{z}\|_2$ for any unitary $\mathbf{U}$.
- One can show that unitary matrices also do not change matrix norms $\|\cdot\|_2$ and $\|\cdot\|_F$, i.e. for any square $\mathbf{A}$ and unitary $\mathbf{U}, \mathbf{V}$:

$$\|\mathbf{U}\mathbf{A}\mathbf{V}\|_2 = \|\mathbf{A}\|_2 \quad \|\mathbf{U}\mathbf{A}\mathbf{V}\|_F = \|\mathbf{A}\|_F.$$

- For $\|\cdot\|_2$ it follows from the definition of an operator norm and the fact that vector 2-norm is unitary invariant.
- For $\|\cdot\|_F$ it follows from $\|\mathbf{A}\|_F^2 = \text{trace}(\mathbf{A}^* \mathbf{A})$ and the fact that $\text{trace}(\mathbf{BC}) = \text{trace}(\mathbf{CB})$.

Householder matrices

- Householder matrix is the matrix of the form

$$\mathbf{H} \equiv \mathbf{H}(\mathbf{v}) = \mathbf{I} - 2\mathbf{v}\mathbf{v}^*,$$

where $\mathbf{v}$ is an $n \times 1$ column and $\mathbf{v}^*\mathbf{v} = 1$ (i.e., $\mathbf{v}$ is a unit vector).

- Can you show that $\mathbf{H}$ is **unitary** and **Hermitian** ($\mathbf{H}^* = \mathbf{H}$)?
- It is also a reflection:

$$\mathbf{H}\mathbf{x} = \mathbf{x} - 2(\mathbf{v}^*\mathbf{x})\mathbf{v}$$

Proof Sketch: $\mathbf{H}$ is Unitary and Hermitian

1. Proof that $\mathbf{H}$ is Hermitian ($\mathbf{H}^* = \mathbf{H}$):

A matrix is Hermitian if it is equal to its own conjugate transpose.

$$\mathbf{H}^* = (\mathbf{I} - 2\mathbf{v}\mathbf{v}^*)^*$$

$$\mathbf{H}^* = \mathbf{I}^* - 2(\mathbf{v}\mathbf{v}^*)^*$$

$$\mathbf{H}^* = \mathbf{I} - 2((\mathbf{v}^*)^*\mathbf{v}^*) \quad (\text{Since } (\mathbf{AB})^* = \mathbf{B}^*\mathbf{A}^* \text{ and } (\mathbf{v}^*)^* = \mathbf{v})$$

$$\mathbf{H}^* = \mathbf{I} - 2\mathbf{v}\mathbf{v}^*$$

$$\mathbf{H}^* = \mathbf{H}$$

Therefore, $\mathbf{H}$ is Hermitian.

2. Proof that $\mathbf{H}$ is Unitary ($\mathbf{H}^*\mathbf{H} = \mathbf{I}$):

A matrix is unitary if $\mathbf{H}^*\mathbf{H} = \mathbf{I}$. Since $\mathbf{H}^* = \mathbf{H}$, we check $\mathbf{H}\mathbf{H} = \mathbf{I}$.

$$\mathbf{H}\mathbf{H} = (\mathbf{I} - 2\mathbf{v}\mathbf{v}^*)(\mathbf{I} - 2\mathbf{v}\mathbf{v}^*)$$

$$\mathbf{H}^2 = \mathbf{I}\mathbf{I} - 2\mathbf{v}\mathbf{v}^*\mathbf{I} - \mathbf{I}2\mathbf{v}\mathbf{v}^* + 4(\mathbf{v}\mathbf{v}^*)(\mathbf{v}\mathbf{v}^*)$$

$$\mathbf{H}^2 = \mathbf{I} - 4\mathbf{v}\mathbf{v}^* + 4\mathbf{v}(\mathbf{v}^*\mathbf{v})\mathbf{v}^*$$

Now, we use the property that $\mathbf{v}^*\mathbf{v} = 1$:

$$\mathbf{H}^2 = \mathbf{I} - 4\mathbf{v}\mathbf{v}^* + 4\mathbf{v}(1)\mathbf{v}^*$$

$$\mathbf{H}^2 = \mathbf{I} - 4\mathbf{v}\mathbf{v}^* + 4\mathbf{v}\mathbf{v}^*$$

$$\mathbf{H}^2 = \mathbf{I}$$

Since $\mathbf{H}\mathbf{H} = \mathbf{I}$, and $\mathbf{H}^* = \mathbf{H}$, $\mathbf{H}$ is unitary.*

Example: Conceptual Reflection

The Householder transformation $\mathbf{H}\mathbf{x} = \mathbf{x} - 2(\mathbf{v}^*\mathbf{x})\mathbf{v}$ is a reflection across the hyperplane orthogonal to $\mathbf{v}$.

Consider the real case where the vector $\mathbf{v}$ is:

$$\mathbf{v} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

1. Reflection of $\mathbf{v}$ itself:

$$\mathbf{H}\mathbf{v} = \mathbf{v} - 2(\mathbf{v}^T\mathbf{v})\mathbf{v} = \mathbf{v} - 2(1)\mathbf{v} = -\mathbf{v}$$

The vector $\mathbf{v}$ is mapped to its negative, i.e., it is reflected through the origin.

2. Reflection of a perpendicular vector $\mathbf{z}$ (where $\mathbf{v}^T\mathbf{z} = 0$):

$$\mathbf{H}\mathbf{z} = \mathbf{z} - 2(\mathbf{v}^T\mathbf{z})\mathbf{v} = \mathbf{z} - 2(0)\mathbf{v} = \mathbf{z}$$

Any vector perpendicular to $\mathbf{v}$ is unchanged, as it lies in the plane of reflection.

Important property of Householder reflection

- A nice property of Householder transformation is that it can zero all elements of a vector except for the first one:

$$\mathbf{H} \begin{pmatrix} \times \\ \times \\ \times \\ \times \end{pmatrix} = \begin{pmatrix} \times \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad \text{i.e.} \quad \mathbf{H}\mathbf{x} = \alpha\mathbf{e}_1.$$

Proof (for real case). Let $\mathbf{e}_1 = (1, 0, \dots, 0)^T$, then we want to find $\mathbf{v}$ such that

$$\mathbf{H}\mathbf{x} = \mathbf{x} - 2(\mathbf{v}^T\mathbf{x})\mathbf{v} = \alpha\mathbf{e}_1,$$

where α is an unknown constant. Since $\|\cdot\|_2$ is unitary invariant we get

$$\|\mathbf{x}\|_2 = \|\mathbf{H}\mathbf{x}\|_2 = \|\alpha\mathbf{e}_1\|_2 = |\alpha|\|\mathbf{e}_1\|_2 = |\alpha|.$$

This gives the value of α :

$$\alpha = \pm\|\mathbf{x}\|_2$$

Also, we can express $\mathbf{v}$ from the target equation $\mathbf{x} - 2(\mathbf{v}^T\mathbf{x})\mathbf{v} = \alpha\mathbf{e}_1$:

$$2(\mathbf{v}^T\mathbf{x})\mathbf{v} = \mathbf{x} - \alpha\mathbf{e}_1$$

$$\mathbf{v} = \frac{\mathbf{x} - \alpha \mathbf{e}_1}{2(\mathbf{v}^T \mathbf{x})}$$

Multiplying the above expression by $\mathbf{v}^T$ (which is $\mathbf{v}^*$ in the general case) we get the same as multiplying by $\mathbf{x}^*$:

$$\mathbf{x}^T \mathbf{x} - 2(\mathbf{v}^T \mathbf{x}) \mathbf{x}^T \mathbf{v} = \alpha \mathbf{x}^T \mathbf{e}_1;$$

$$\|\mathbf{x}\|_2^2 - 2(\mathbf{v}^T \mathbf{x})^2 = \alpha x_1$$

(Note:

$$\mathbf{x}^T \mathbf{e}_1 = x_1$$

, the first component of $\mathbf{x}$)

Finalizing the value of $\mathbf{v}^T \mathbf{x}$:

From the last equation, we can solve for $(\mathbf{v}^T \mathbf{x})^2$:

$$2(\mathbf{v}^T \mathbf{x})^2 = \|\mathbf{x}\|_2^2 - \alpha x_1$$

$$\mathbf{v}^T \mathbf{x} = \pm \sqrt{\frac{\|\mathbf{x}\|_2^2 - \alpha x_1}{2}}$$

Finally, by substituting this back into the expression for $\mathbf{v}$ (and noting $\mathbf{v}$ must be a unit vector, $\mathbf{v} = \frac{\mathbf{x} - \alpha \mathbf{e}_1}{\|\mathbf{x} - \alpha \mathbf{e}_1\|_2}$), the Householder vector $\mathbf{v}$ required to perform the transformation is:

$$\mathbf{v} = \frac{\mathbf{x} - \alpha \mathbf{e}_1}{\|\mathbf{x} - \alpha \mathbf{e}_1\|_2}, \quad \text{where } \alpha = -\text{sgn}(x_1) \|\mathbf{x}\|_2$$

(Note: The sign of α is chosen to avoid cancellation error in $\mathbf{x} - \alpha \mathbf{e}_1$)

Small Example

Let's use a real vector $\mathbf{x}$ to be zeroed:

$$\mathbf{x} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

1. Determine α :

$$\|\mathbf{x}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = 5$$

We choose $\alpha = -\|\mathbf{x}\|_2 = -5$ (choosing the negative sign often provides better numerical stability).

2. Determine the target vector $\alpha \mathbf{e}_1$:

$$\mathbf{H}\mathbf{x} = \alpha \mathbf{e}_1 = -5 \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} -5 \\ 0 \end{pmatrix}$$

The transformation successfully zeros the second component.

3. Determine the Householder vector $\mathbf{v}$:

The un-normalized vector is $\mathbf{u} = \mathbf{x} - \alpha \mathbf{e}_1$:

$$\mathbf{u} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} - (-5) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3+5 \\ 4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \end{pmatrix}$$

Now, normalize it to find $\mathbf{v}$:

$$\|\mathbf{u}\|_2 = \sqrt{8^2 + 4^2} = \sqrt{64 + 16} = \sqrt{80}$$

$$\mathbf{v} = \frac{1}{\sqrt{80}} \begin{pmatrix} 8 \\ 4 \end{pmatrix}$$

(This vector $\mathbf{v}$ generates the Householder matrix $\mathbf{H} = \mathbf{I} - 2\mathbf{v}\mathbf{v}^T$ that performs the desired transformation).

Givens (Jacobi) matrix

- A Givens matrix is a matrix

$$\mathbf{G} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix},$$

which is a rotation.

- For a general case, we select two (i, j) planes and rotate vector $\mathbf{x}$

$$\mathbf{x}' = \mathbf{G}\mathbf{x},$$

only in the i -th and j -th positions.

The components transform as:

$$x'_i = x_i \cos \alpha - x_j \sin \alpha, \quad x'_j = x_i \sin \alpha + x_j \cos \alpha,$$

with all other x_k remain unchanged ($x'_k = x_k$ for $k \neq i, j$).

- Therefore, we can make elements in the j -th position zero ($x'_j = 0$) by choosing α such that:

$$\cos \alpha = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}, \quad \sin \alpha = -\frac{x_j}{\sqrt{x_i^2 + x_j^2}}$$

Small Example: Zeroing a Component

Let's use a $2D$ vector $\mathbf{x}$ and aim to zero out the second component ($j = 2$):

$$\mathbf{x} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}, \quad i = 1, j = 2$$

1. Calculate the magnitude (denominator):

$$r = \sqrt{x_i^2 + x_j^2} = \sqrt{3^2 + 4^2} = 5$$

2. Determine $\cos \alpha$ and $\sin \alpha$ (The parameters for G):

$$\cos \alpha = \frac{x_i}{r} = \frac{3}{5}$$

$$\sin \alpha = -\frac{x_j}{r} = -\frac{4}{5}$$

3. Construct the Givens matrix G :

$$\mathbf{G} = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} = \begin{pmatrix} 3/5 & -(-4/5) \\ -4/5 & 3/5 \end{pmatrix} = \begin{pmatrix} 0.6 & 0.8 \\ -0.8 & 0.6 \end{pmatrix}$$

4. Perform the rotation $\mathbf{x}' = G\mathbf{x}$:

$$\mathbf{x}' = \begin{pmatrix} 0.6 & 0.8 \\ -0.8 & 0.6 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$

- $x'_1 = (0.6)(3) + (0.8)(4) = 1.8 + 3.2 = 5$
- $x'_2 = (-0.8)(3) + (0.6)(4) = -2.4 + 2.4 = 0$

$$\mathbf{x}' = \begin{pmatrix} 5 \\ 0 \end{pmatrix}$$

The Givens rotation successfully zeroed the second component and preserved the 2-norm: $\|\mathbf{x}\|_2 = 5$ and $\|\mathbf{x}'\|_2 = 5$.

Singular Value Decomposition

Theorem. Any matrix $\mathbf{A} \in \mathbb{C}^{n \times m}$ can be written as a product of three matrices:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*,$$

where

- $\mathbf{U}$ is an $n \times n$ unitary matrix

- $\mathbf{V}$ is an $m \times m$ unitary matrix
- $\mathbf{\Sigma}$ is a diagonal matrix with non-negative elements $\sigma_1 \geq \dots \geq \sigma_{\min(n,m)}$ on the diagonal.

Conceptual Instructions: How SVD is Found

The SVD is found through an eigenvalue decomposition process on a related Hermitian matrix.

- 1. Form the Hermitian Matrix:** Calculate $\mathbf{M} = \mathbf{A}^* \mathbf{A}$.
 - 2. Find $\mathbf{V}$ (Right Singular Vectors):** Find the orthonormal eigenvectors of $\mathbf{M}$. These vectors form the columns of the unitary matrix $\mathbf{V}$.
 - 3. Find $\mathbf{\Sigma}$ (Singular Values):** The eigenvalues (λ_i) of $\mathbf{M}$ are the squares of the singular values ($\sigma_i = \sqrt{\lambda_i}$). Construct the diagonal matrix $\mathbf{\Sigma}$ by placing σ_i on the diagonal, sorted from largest to smallest.
 - 4. Find $\mathbf{U}$ (Left Singular Vectors):** Use the relation $\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{A} \mathbf{v}_i$. These vectors form the columns of the unitary matrix $\mathbf{U}$.
-

Small Example: 2×2 Matrix SVD

Let's find the SVD for the following matrix:

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

Step 1 & 2: Find $\mathbf{V}$ and $\mathbf{\Sigma}$ (via $\mathbf{A}^T \mathbf{A}$)

Since $\mathbf{A}$ is real, $\mathbf{A}^* = \mathbf{A}^T$.

$$\mathbf{M} = \mathbf{A}^T \mathbf{A} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

Eigenvalues of $\mathbf{M}$:

The characteristic equation is $\det(\mathbf{M} - \lambda \mathbf{I}) = (1 - \lambda)^2 - 1 = 0$, which simplifies to $\lambda^2 - 2\lambda = 0$. The eigenvalues are $\lambda_1 = 2$ and $\lambda_2 = 0$.

Singular Values ($\mathbf{\Sigma}$):

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{2}$$

$$\sigma_2 = \sqrt{\lambda_2} = 0$$

$$\Sigma = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix}$$

Note: This immediately tells us $\text{rank}(\mathbf{A}) = 1$ (the number of non-zero singular values).

Right Singular Vectors (V):

- For $\lambda_1 = 2$: Eigenvector $\mathbf{v}_1$: $\begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \mathbf{v}_1 = \mathbf{0} \implies \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$
- For $\lambda_2 = 0$: Eigenvector $\mathbf{v}_2$: $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \mathbf{v}_2 = \mathbf{0} \implies \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$

$$\mathbf{V} = (\mathbf{v}_1 \quad \mathbf{v}_2) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Step 3: Find U (Left Singular Vectors)

Use $\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{A} \mathbf{v}_i$:

- For $\sigma_1 = \sqrt{2}$:

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

- For $\sigma_2 = 0$: The method breaks down, so $\mathbf{u}_2$ is chosen to be orthogonal to $\mathbf{u}_1$:
 $\mathbf{u}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

$$\mathbf{U} = (\mathbf{u}_1 \quad \mathbf{u}_2) = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Final SVD Result

$$\mathbf{A} = \mathbf{U} \Sigma \mathbf{V}^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

(Note: $\mathbf{U} = \mathbf{I}$ here, as the transformation $\mathbf{A} \mathbf{v}_1$ already points along the first standard basis vector $\mathbf{e}_1$.)

Proof Sketch for Singular Value Decomposition (SVD)

The proof relies on properties of the matrix $\mathbf{A}^* \mathbf{A}$.

- **Property 1 (Hermitian/Diagonalizable):** Matrix $\mathbf{A}^* \mathbf{A}$ is Hermitian, hence diagonalizable in a unitary basis.
- **Property 2 (Non-Negative Definite):** $\mathbf{A}^* \mathbf{A} \geq 0$ (non-negative definite), so its eigenvalues are non-negative.
- **Eigen-Decomposition:** Therefore, there exists a unitary matrix $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_n]$ such that:

$$\mathbf{V}^* \mathbf{A}^* \mathbf{A} \mathbf{V} = \text{diag}(\sigma_1^2, \dots, \sigma_n^2), \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n.$$

Construction of the SVD

1. Define Rank (r) and Truncated Matrices:

Let $\sigma_i = 0$ for $i > r$, where $r = \text{rank}(\mathbf{A})$.

Let $\mathbf{V}_r = [\mathbf{v}_1, \dots, \mathbf{v}_r]$ (columns corresponding to non-zero σ_i), and

$\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r)$.

2. Define $\mathbf{U}_r$ (The Core of the Left Unitary Matrix):

From the eigen-decomposition, we can show:

$$\mathbf{V}_r^* \mathbf{A}^* \mathbf{A} \mathbf{V}_r = \Sigma_r^2 \implies (\Sigma_r^{-1} \mathbf{V}_r^* \mathbf{A}^*)(\mathbf{A} \mathbf{V}_r \Sigma_r^{-1}) = \mathbf{I}.$$

Comment: This proves that the columns of the matrix $\mathbf{U}_r = \mathbf{A} \mathbf{V}_r \Sigma_r^{-1}$ are orthonormal, since $\mathbf{U}_r^ \mathbf{U}_r = \mathbf{I}$.*

3. Completing $\mathbf{U}$:

Let us add to $\mathbf{U}_r$ any orthogonal columns (that are orthogonal to columns in $\mathbf{U}_r$) and denote this matrix as the full unitary matrix $\mathbf{U}$.

4. Final Identity:

This construction leads to the fundamental relationship $\mathbf{A} \mathbf{V} = \mathbf{U} \begin{pmatrix} \Sigma_r & 0 \\ 0 & 0 \end{pmatrix}$.

Multiplying by $\mathbf{U}^*$ gives the decomposition:

$$\mathbf{A} \mathbf{V} = \mathbf{U} \begin{pmatrix} \Sigma_r & 0 \\ 0 & 0 \end{pmatrix} \implies \mathbf{U}^* \mathbf{A} \mathbf{V} = \begin{pmatrix} \Sigma_r & 0 \\ 0 & 0 \end{pmatrix}.$$

(Since $\mathbf{U}$ and $\mathbf{V}$ are non-singular unitary matrices, the rank of the matrix $\mathbf{A}$ is equal to the rank of Σ , so $r = \text{rank}(\mathbf{A})$).

Corollaries of the SVD

Corollary 1: The Dyadic Expansion (Low-Rank Sum)

The SVD can be written as a sum of rank-one matrices, providing the best low-rank approximation:

$$\mathbf{A} = \sum_{\alpha=1}^r \sigma_{\alpha} \mathbf{u}_{\alpha} \mathbf{v}_{\alpha}^*$$

or element-wise

$$a_{ij} = \sum_{\alpha=1}^r \sigma_{\alpha} u_{i\alpha} \bar{v}_{j\alpha}.$$

Corollary 2: Fundamental Subspaces (The Four Subspaces)

The SVD directly gives the basis for all four fundamental subspaces of a matrix.

Subspace	Definition	Basis Spanned by SVD Vectors
Kernel / Nullspace	$\ker(\mathbf{A})$	$\mathcal{L}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$
Image / Column Space	$\text{im}(\mathbf{A})$	$\mathcal{L}\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$
Left Nullspace	$\ker(\mathbf{A}^*)$	$\mathcal{L}\{\mathbf{u}_{r+1}, \dots, \mathbf{u}_n\}$
Row Space	$\text{im}(\mathbf{A}^*)$	$\mathcal{L}\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$

(Comment: This shows that the basis for the four subspaces are the columns of $\mathbf{U}$ and $\mathbf{V}$, partitioned by the rank r .)

Linear dependence

Definition. Vectors $\mathbf{a}_i$ are called **linearly dependent**, if there exist simultaneously non-zero coefficients x_i such that

$$\sum_i \mathbf{a}_i x_i = \mathbf{0},$$

or in the matrix form

$$\mathbf{A}\mathbf{x} = \mathbf{0}, \quad \|\mathbf{x}\| \neq 0.$$

In this case, we say that the matrix $\mathbf{A}$ has a non-trivial **nullspace** (or **kernel**) denoted by $\mathcal{N}(\mathbf{A})$ (or $\ker(\mathbf{A})$).

Vectors that are not linearly dependent are called **linearly independent**.

Example: Linear Dependence and Nullspace

Consider the matrix $\mathbf{A}$ formed by two column vectors $\mathbf{a}_1$ and $\mathbf{a}_2$:

$$\mathbf{A} = (\mathbf{a}_1 \quad \mathbf{a}_2) = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$$

We find a non-zero solution to $\mathbf{A}\mathbf{x} = \mathbf{0}$ with $\mathbf{x} = \begin{pmatrix} -2 \\ 1 \end{pmatrix}$.

- **Linear Dependence:** Since $\mathbf{x} \neq \mathbf{0}$ solves $\mathbf{A}\mathbf{x} = \mathbf{0}$, the vectors $\mathbf{a}_1$ and $\mathbf{a}_2$ are **linearly dependent** (as $(-2)\mathbf{a}_1 + (1)\mathbf{a}_2 = \mathbf{0}$).
- **Nullspace:** The non-trivial nullspace $\mathcal{N}(\mathbf{A})$ is the line $\text{span} \left\{ \begin{pmatrix} -2 \\ 1 \end{pmatrix} \right\}$.

Example: Linear Independence

For $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, the only solution to $\mathbf{A}\mathbf{x} = \mathbf{0}$ is $\mathbf{x} = \mathbf{0}$.

- **Linear Independence:** The column vectors are **linearly independent**.
 - **Nullspace:** The nullspace is trivial: $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$.
-

Linear (vector) space

A **linear space** spanned by vectors $\{\mathbf{a}_1, \dots, \mathbf{a}_m\}$ is defined as all possible vectors of the form

$$\mathcal{L}(\mathbf{a}_1, \dots, \mathbf{a}_m) = \left\{ \mathbf{y} : \mathbf{y} = \sum_{i=1}^m \mathbf{a}_i x_i, \forall \mathbf{x} \in \mathbb{R}^m \right\},$$

In the matrix form, the linear space is a set of all $\mathbf{y}$ such that

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

This set is also called the **range** (or **image**) of the matrix, denoted by $\text{range}(\mathbf{A})$ (or $\text{im}(\mathbf{A})$) respectively.

Example: Linear Space (Range/Image)

Using the same dependent matrix $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$:

$$\mathbf{y} = \mathbf{A}\mathbf{x} = (x_1 + 2x_2) \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

- The **range** $\text{range}(\mathbf{A})$ is the set of all vectors that are scalar multiples of $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$. This space is a **line passing through the origin in $\mathbb{R}^2$** , which is a valid linear space.

Example: Set that is NOT a Linear Space (Non-Linear Space Concept)

A set is **NOT** a linear space if it is not closed under addition or scalar multiplication, or if it does not contain the origin.

- **Set:** The vectors lying on the unit circle: $S = \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x}\| = 1\}$.
- **Reason:** Take $\mathbf{x}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $\mathbf{x}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ (both in S). Their sum is $\mathbf{x}_1 + \mathbf{x}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$. Since $\|\mathbf{x}_1 + \mathbf{x}_2\| = \sqrt{2} \neq 1$, the sum is **not** in S . Therefore, S is **not a linear space**.

Dimension of a linear space

- The dimension of a linear space $\text{im}(\mathbf{A})$ denoted by $\dim \text{im}(\mathbf{A})$ is the minimal number of vectors required to represent each vector from $\text{im}(\mathbf{A})$.
- The dimension of $\text{im}(\mathbf{A})$ has a direct connection to the **matrix rank**.

Matrix rank

- **Rank of a matrix $\mathbf{A}$** is a maximal number of linearly independent **columns** in a matrix $\mathbf{A}$, or the **dimension of its column space** $= \dim \text{im}(\mathbf{A})$.
- You can also use linear combination of **rows** to define the rank, i.e. formally there are two ranks: column rank and row rank of a matrix.

Example: Calculating Matrix Rank

Consider the 2×3 matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$$

1. Column Rank (Linear independence of columns):

- Column 2 is $2 \times$ Column 1. Column 3 is $3 \times$ Column 1.

- The maximum number of linearly independent columns is **1** (e.g., just the first column).
- Thus, $\text{rank}(\mathbf{A}) = 1$.

2. Row Rank (Linear independence of rows):

- Row 2 is $2 \times$ Row 1.
- The maximum number of linearly independent rows is **1** (e.g., just the first row).
- Thus, the row rank is also **1**.

The example demonstrates that for this matrix, the row rank equals the column rank.

Theorem: Row Rank equals Column Rank

The dimension of the column space of the matrix is equal to the dimension of its row space.

- In the matrix form this fact can be written as $\dim \text{im}(\mathbf{A}) = \dim \text{im}(\mathbf{A}^T)$.
- Thus, there is a single rank!

Proof Sketch (Using Singular Value Decomposition)

The proof that row rank equals column rank is an elegant consequence of the **Singular Value Decomposition (SVD)**, which we just covered: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$.

- 1. Rank via SVD:** The rank of $\mathbf{A}$ is equal to the number of non-zero singular values in $\mathbf{\Sigma}$. Let this number be r .
- 2. Column Rank:** The column space $\text{im}(\mathbf{A})$ is spanned by the first r columns of the matrix $\mathbf{U}$ (the left singular vectors), as these correspond to the non-zero singular values. Thus, $\dim \text{im}(\mathbf{A}) = r$.
- 3. Row Rank:** The row space $\text{im}(\mathbf{A}^T)$ is the same as the column space of $\mathbf{A}^T$ (or $\mathbf{A}^*$). Since $\mathbf{A}^* = \mathbf{V}\mathbf{\Sigma}^*\mathbf{U}^*$, the column space of $\mathbf{A}^*$ is spanned by the first r columns of $\mathbf{V}$ (the right singular vectors). Thus, $\dim \text{im}(\mathbf{A}^T) = r$.

Since both the dimension of the column space and the dimension of the row space are equal to r (the number of non-zero singular values), **Row Rank = Column Rank**.

Johnson–Lindenstrauss lemma

Let $N \gg 1$. Given $0 < \epsilon < 1$, a set of m points in $\mathbb{R}^N$ and $n > \frac{8 \log m}{\epsilon^2}$ (we want $n \ll N$).

Then there exists a linear map f from $\mathbb{R}^N \rightarrow \mathbb{R}^n$ such that the following inequality holds:

$$(1 - \epsilon)\|\mathbf{u} - \mathbf{v}\|^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|^2 \leq (1 + \epsilon)\|\mathbf{u} - \mathbf{v}\|^2.$$

- This theorem states that there exists a map from a high- to a low-dimensional space so that distances between points in these spaces are **almost the same**.
 - It is not very practical due to the dependence on ϵ .
 - This lemma does not give a recipe how to construct f , but guarantees that f exists.
-

Example: Conceptual Impact of the Johnson–Lindenstrauss Lemma

The JL Lemma's value lies in showing how the required dimensionality of the new space (n) depends *logarithmically* on the number of points (m), but is *independent* of the original dimensionality (N).

Scenario: We have a dataset with $m = 1,000,000$ data points, each in a very high-dimensional space $\mathbb{R}^N$ where $N = 10,000$. We are willing to accept a distance distortion of $\epsilon = 0.1$ (a 10% error).

1. Calculate Required New Dimension (n):

$$n > \frac{8 \log m}{\epsilon^2}$$

- $m = 10^6 \implies \log(m) = \ln(10^6) \approx 13.82$
- $\epsilon = 0.1 \implies \epsilon^2 = 0.01$

$$n > \frac{8 \cdot 13.82}{0.01} = 11,056$$

2. Analyze the Result:

- The original space had $N = 10,000$ dimensions (we assumed N could be much larger, but let's use 10,000 for a baseline).
- The required new dimension is $n \approx 11,056$.

3. The Crucial Insight (Where the Lemma shines):

- **The Power of m :** If we kept $m = 10^6$ but increased the original dimension to $N = 1,000,000$, the new dimension n would **remain the same** ($\approx 11,056$).
- **The Guarantee:** The lemma guarantees that a linear projection exists that takes our 1,000,000 points from 1,000,000 dimensions down to just $\approx 11,056$ dimensions, with

the guarantee that **all pairwise distances are preserved within 10% of their original value.**

This logarithmic dependence on the number of points is what makes the lemma theoretically vital for proving the effectiveness of random projections in massive datasets.

Proof for the skeleton decomposition

- Let $\mathbf{C} \in \mathbb{C}^{n \times k}$ be the k columns based on the nonsingular submatrix $\hat{\mathbf{A}}$. Therefore they are linearly independent.
- Take any other column $\mathbf{a}_i$ of $\mathbf{A}$. Then $\mathbf{a}_i$ can be represented as a linear combination of the columns of $\mathbf{C}$, i.e., $\mathbf{a}_i = \mathbf{C}\mathbf{x}_i$, where $\mathbf{x}_i$ is a vector of coefficients.
- $\mathbf{a}_i = \mathbf{C}\mathbf{x}_i$ are n equations. We take k equations of those corresponding to the rows that contain $\hat{\mathbf{A}}$ and get the equation

$$\hat{\mathbf{r}}_i = \hat{\mathbf{A}}\mathbf{x}_i \implies \mathbf{x}_i = \hat{\mathbf{A}}^{-1}\hat{\mathbf{r}}_i$$

- Thus, $\mathbf{a}_i = \mathbf{C}\hat{\mathbf{A}}^{-1}\hat{\mathbf{r}}_i$ for every i and

$$\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_m] = \mathbf{C}\hat{\mathbf{A}}^{-1}\mathbf{R}.$$

(Where $\mathbf{R}$ is the matrix containing $\hat{\mathbf{r}}_i$ for all columns, effectively corresponding to the rows of $\mathbf{A}$)

Function interpolation with CUR

If a matrix $\mathbf{A}$ contains values of a function $f(x, y)$ at points $\{x_i\}_{i=1}^N$ and $\{y_j\}_{j=1}^M$, then skeleton decomposition $\mathbf{A} = \mathbf{C}\hat{\mathbf{A}}^{-1}\mathbf{R}$ helps to construct an interpolation:

$$f(x, y) \approx (f(x, \hat{y}_1) \quad \dots \quad f(x, \hat{y}_r)) \begin{pmatrix} f(\hat{x}_1, \hat{y}_1) & \dots & f(\hat{x}_r, \hat{y}_1) \\ \vdots & \ddots & \vdots \\ f(\hat{x}_r, \hat{y}_r) & \dots & f(\hat{x}_r, \hat{y}_r) \end{pmatrix}^{-1} \begin{pmatrix} f(\hat{x}_1, y) \\ \vdots \\ f(\hat{x}_r, y) \end{pmatrix}$$

Conceptual Summary and Application

The **Skeleton/CUR Decomposition** is a low-rank matrix approximation ($\mathbf{A} \approx \mathbf{CUR}$) that uses actual columns ($\mathbf{C}$) and rows ($\mathbf{R}$) from the original matrix $\mathbf{A}$, where $\mathbf{U}$ (here $\hat{\mathbf{A}}^{-1}$) is a small matrix derived from the intersection of these columns and rows.

Purpose of the Decomposition:

- It proves that **every** column $\mathbf{a}_i$ and **every** row $\mathbf{r}_j^T$ of $\mathbf{A}$ is a linear combination of a small, chosen set of k independent columns ($\mathbf{C}$) and rows ($\mathbf{R}$).
- It is a more **interpretable** low-rank approximation than SVD, as the vectors $\mathbf{C}$ and $\mathbf{R}$ correspond to actual data points/features, not abstract eigenvectors.

Purpose of the Interpolation Formula (Using $\mathbf{A} = \mathbf{C}\hat{\mathbf{A}}^{-1}\mathbf{R}$):

- The interpolation formula shown is a method to estimate an unknown function value $f(x, y)$ by leveraging the low-rank structure of the data matrix.
- **Input Data:** The matrix $\mathbf{A}$ represents the function values $f(x_i, y_j)$ at a grid of known points.
- **Interpolation:** The formula essentially takes known values *along a slice* (the r chosen columns and r chosen rows, which pass through the $\hat{\mathbf{A}}$ submatrix) and uses them to construct an approximation of $f(x, y)$ at an arbitrary point (x, y) .
- **Intuition:** The function is approximated by a linear combination of its values on a small, carefully chosen grid, providing an efficient way to reconstruct the entire function surface.

Proof Sketch for Singular Value Decomposition (SVD)

The proof relies on properties of the matrix $\mathbf{A}^*\mathbf{A}$.

- **Property 1 (Hermitian/Diagonalizable):** Matrix $\mathbf{A}^*\mathbf{A}$ is Hermitian, hence diagonalizable in a unitary basis.
- **Property 2 (Non-Negative Definite):** $\mathbf{A}^*\mathbf{A} \geq 0$ (non-negative definite), so its eigenvalues are non-negative.
- **Eigen-Decomposition:** Therefore, there exists a unitary matrix $\mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_n]$ such that:

$$\mathbf{V}^*\mathbf{A}^*\mathbf{A}\mathbf{V} = \text{diag}(\sigma_1^2, \dots, \sigma_n^2), \quad \sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n.$$

Construction of the SVD

1. Define Rank (r) and Truncated Matrices:

Let $\sigma_i = 0$ for $i > r$, where $r = \text{rank}(\mathbf{A})$.

Let $\mathbf{V}_r = [\mathbf{v}_1, \dots, \mathbf{v}_r]$ (columns corresponding to non-zero σ_i), and

$\mathbf{\Sigma}_r = \text{diag}(\sigma_1, \dots, \sigma_r)$.

2. Define $\mathbf{U}_r$ (The Core of the Left Unitary Matrix):

From the eigen-decomposition, we can show:

$$\mathbf{V}_r^* \mathbf{A}^* \mathbf{A} \mathbf{V}_r = \Sigma_r^2 \implies (\Sigma_r^{-1} \mathbf{V}_r^* \mathbf{A}^*)(\mathbf{A} \mathbf{V}_r \Sigma_r^{-1}) = \mathbf{I}.$$

Comment: This proves that the columns of the matrix $\mathbf{U}_r = \mathbf{A} \mathbf{V}_r \Sigma_r^{-1}$ are orthonormal, since $\mathbf{U}_r^ \mathbf{U}_r = \mathbf{I}$.*

3. Completing $\mathbf{U}$:

Let us add to $\mathbf{U}_r$ any orthogonal columns (that are orthogonal to columns in $\mathbf{U}_r$) and denote this matrix as the full unitary matrix $\mathbf{U}$.

4. Final Identity:

This construction leads to the fundamental relationship $\mathbf{A} \mathbf{V} = \mathbf{U} \begin{pmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$.

Multiplying by $\mathbf{U}^*$ gives the decomposition:

$$\mathbf{A} \mathbf{V} = \mathbf{U} \begin{pmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \implies \mathbf{U}^* \mathbf{A} \mathbf{V} = \begin{pmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}.$$

(Since $\mathbf{U}$ and $\mathbf{V}$ are non-singular unitary matrices, the rank of the matrix $\mathbf{A}$ is equal to the rank of Σ , so $r = \text{rank}(\mathbf{A})$).

Corollaries of the SVD

Corollary 1: The Dyadic Expansion (Low-Rank Sum)

The SVD can be written as a sum of rank-one matrices, providing the best low-rank approximation:

$$\mathbf{A} = \sum_{\alpha=1}^r \sigma_{\alpha} \mathbf{u}_{\alpha} \mathbf{v}_{\alpha}^*$$

or element-wise

$$a_{ij} = \sum_{\alpha=1}^r \sigma_{\alpha} u_{i\alpha} \bar{v}_{j\alpha}.$$

Corollary 2: Fundamental Subspaces (The Four Subspaces)

The SVD directly gives the basis for all four fundamental subspaces of a matrix.

Subspace	Definition	Basis Spanned by SVD Vectors
Kernel / Nullspace	$\ker(\mathbf{A})$	$\mathcal{L}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$
Image / Column Space	$\text{im}(\mathbf{A})$	$\mathcal{L}\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$
Left Nullspace	$\ker(\mathbf{A}^*)$	$\mathcal{L}\{\mathbf{u}_{r+1}, \dots, \mathbf{u}_n\}$
Row Space	$\text{im}(\mathbf{A}^*)$	$\mathcal{L}\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$

(Comment: This shows that the basis for the four subspaces are the columns of $\mathbf{U}$ and $\mathbf{V}$, partitioned by the rank r .)

Main result on low-rank approximation

Corollary: Computation of the best rank- r approximation $\mathbf{A}_r$ is equivalent to setting $\sigma_{r+1} = 0, \dots, \sigma_K = 0$ in the SVD. The error is:

$$\min_{\mathbf{A}_r} \|\mathbf{A} - \mathbf{A}_r\|_2 = \sigma_{r+1},$$

$$\min_{\mathbf{A}_r} \|\mathbf{A} - \mathbf{A}_r\|_F = \sqrt{\sigma_{r+1}^2 + \dots + \sigma_K^2}$$

where $K = \min(n, m)$.

That is why it is important to look at the **decay of the singular values**.

Example: Using Singular Value Decay for Rank Selection

In practice, the rank of a matrix is not a simple integer due to floating-point errors; it is determined by where the singular values "fall off" or decay to near zero.

Scenario: We have a large matrix $\mathbf{A}$ with the following top singular values:

Σ : 100.0, 50.0, 10.0, 0.5, 0.01, 0.0001, ...

Singular Value (σ_i)	Value	Decay Behavior
σ_1	100.0	Primary component
σ_2	50.0	Primary component
σ_3	10.0	Significant component
σ_4	0.5	Small decay
σ_5	0.01	Steep decay (noise level)

Singular Value (σ_i)	Value	Decay Behavior
σ_1	0.5	Meaningful
σ_2	0.01	Meaningful
σ_3	0.0001	Meaningful
σ_4	0.5	Negligible

Using the Corollary to Choose Optimal Rank (r):

1. If we choose Rank $r = 3$:

- $\mathbf{A}_3$ uses $\sigma_1, \sigma_2, \sigma_3$.
- The error is determined by the next value, $\sigma_4 = 0.5$.
- $\|\mathbf{A} - \mathbf{A}_3\|_2 = \sigma_4 = 0.5$ (Spectral Norm Error)
- $\|\mathbf{A} - \mathbf{A}_3\|_F = \sqrt{0.5^2 + 0.01^2 + 0.0001^2 + \dots} \approx \mathbf{0.5}$ (Frobenius Norm Error)
- **Conclusion:** This is a very good approximation, sacrificing only a small amount of information for a significantly reduced matrix size.

2. If we choose Rank $r = 5$:

- $\mathbf{A}_5$ uses σ_1 through σ_5 .
- The error is $\sigma_6 = 0.0001$.
- $\|\mathbf{A} - \mathbf{A}_5\|_2 = \mathbf{0.0001}$
- **Conclusion:** The error is almost zero. This choice captures all meaningful information while ignoring only what is essentially numerical noise.

The "decay of the singular values" plot is used precisely to find the "elbow" where the values drop steeply, telling us the **intrinsic rank** of the data.

Matrix form

The system of linear equations

$$\begin{pmatrix} 2 & 3 & 0 \\ 2 & 0 & 3 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \\ 2 \end{pmatrix}$$

or simply

$$\mathbf{A}\mathbf{u} = \mathbf{f},$$

where $\mathbf{A}$ is a 3×3 matrix and $\mathbf{f}$ is the right-hand side.

Over/under determined linear systems

If the system $\mathbf{A}\mathbf{u} = \mathbf{f}$ has:

- **More equations than unknowns** ($m > n$ equations, n unknowns), it is called **overdetermined** system (generically, no solution).

- **Less equations than unknowns** ($m < n$ equations, n unknowns), it is called **underdetermined** system (solution is non-unique, to make it unique additional assumptions have to be made).

Example: Overdetermined vs. Underdetermined

System Type	Matrix $\mathbf{A}$ Size ($m \times n$)	Visual Example	Solution Status
Overdetermined	3×2	3 lines intersecting at 1 point? (unlikely)	No solution (in general)
Underdetermined	2×3	2 planes intersecting in $\mathbb{R}^3$	Infinite solutions (a line of solutions)

Existence of solutions

A solution to the linear system of equations with a **square matrix $\mathbf{A}$**

$$\mathbf{A}\mathbf{u} = \mathbf{f}$$

exists, **if and only if** (iff):

- $\det \mathbf{A} \neq 0$
- or matrix $\mathbf{A}$ has **full rank** ($\text{rank}(\mathbf{A}) = n$).

Example: Checking the Square Matrix

For the 3×3 example matrix $\mathbf{A} = \begin{pmatrix} 2 & 3 & 0 \\ 2 & 0 & 3 \\ 1 & 1 & 0 \end{pmatrix}$, we check the determinant to see if a unique solution exists:

$$\det(\mathbf{A}) = 2 \begin{vmatrix} 0 & 3 \\ 1 & 0 \end{vmatrix} - 3 \begin{vmatrix} 2 & 3 \\ 1 & 0 \end{vmatrix} + 0$$

$$\det(\mathbf{A}) = 2(0 - 3) - 3(0 - 3)$$

$$\det(\mathbf{A}) = 2(-3) - 3(-3) = -6 + 9 = 3$$

- **Conclusion:** Since $\det(\mathbf{A}) = 3 \neq 0$, the matrix has full rank ($\text{rank}(\mathbf{A}) = 3$). Therefore, a **unique solution** ($\mathbf{x}, \mathbf{y}, \mathbf{z}$) for $\mathbf{A}\mathbf{u} = \mathbf{f}$ **exists**.

Gaussian elimination and LU decomposition

- **Gaussian elimination** is the computation of one of the most important matrix decompositions: **LU-decomposition**.

Definition: LU-decomposition of the square matrix $\mathbf{A}$ is the representation

$$\mathbf{A} = \mathbf{L}\mathbf{U},$$

where

- $\mathbf{L}$ is **lower triangular** (elements strictly above the diagonal are zero)
- $\mathbf{U}$ is **upper triangular** matrix (elements strictly below the diagonal are zero)

This factorization is **non-unique**, so it is typical to require that the matrix $\mathbf{L}$ has ones on the diagonal (this is called **Doolittle decomposition**).

Main goal of the LU decomposition is to solve linear systems ($\mathbf{Ax} = \mathbf{f}$), because

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{f} = (\mathbf{L}\mathbf{U})^{-1}\mathbf{f} = \mathbf{U}^{-1}\mathbf{L}^{-1}\mathbf{f},$$

and this reduces to the solution of two simpler triangular linear systems:

1. **Forward step:** Solve for $\mathbf{y}$:

$$\mathbf{L}\mathbf{y} = \mathbf{f}$$

2. **Backward step:** Solve for $\mathbf{x}$:

$$\mathbf{U}\mathbf{x} = \mathbf{y}$$

Question: Does LU decomposition always exist?

Answer: No. An LU decomposition (without pivoting) does not always exist.

- **Condition:** An LU decomposition exists if and only if all of the matrix's principal leading minors (the determinants of the top-left square submatrices) are non-zero.
- **Failure Example:** The matrix $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ has a zero principal minor ($\det(0) = 0$). You cannot decompose this matrix into $\mathbf{LU}$ without swapping rows.
- **The Solution ($\mathbf{PA}=\mathbf{LU}$):** However, a decomposition of the form $\mathbf{PA} = \mathbf{LU}$ *always* exists, where $\mathbf{P}$ is a **permutation matrix** (representing row swaps, or "pivoting"). This is the standard, reliable method used in practice.

Small Example: Finding LU Decomposition

Find the Doolittle LU decomposition for the matrix:

$$\mathbf{A} = \begin{pmatrix} 4 & 3 \\ 8 & 5 \end{pmatrix}$$

$$\text{Goal: } \mathbf{A} = \mathbf{L}\mathbf{U} \implies \begin{pmatrix} 4 & 3 \\ 8 & 5 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ l_{21} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{pmatrix}$$

Step-by-step Calculation:

1. **First Row:** $1 \cdot u_{11} = 4$, so $u_{11} = 4$. $1 \cdot u_{12} = 3$, so $u_{12} = 3$.
2. **Second Row, First Column:** $l_{21} \cdot u_{11} + 1 \cdot 0 = 8 \implies l_{21} \cdot 4 = 8$, so $l_{21} = 2$.
3. **Second Row, Second Column:**
 $l_{21} \cdot u_{12} + 1 \cdot u_{22} = 5 \implies 2 \cdot 3 + u_{22} = 5 \implies 6 + u_{22} = 5$, so $u_{22} = -1$.

Result:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 4 & 3 \\ 0 & -1 \end{pmatrix}$$

$$(\text{Check: } \mathbf{L}\mathbf{U} = \begin{pmatrix} 4 & 3 \\ 8 & 6-1 \end{pmatrix} = \begin{pmatrix} 4 & 3 \\ 8 & 5 \end{pmatrix} = \mathbf{A})$$

Complexity of the Gaussian elimination/LU decomposition

- Each elimination step requires $\mathcal{O}(n^2)$ operations.
- Thus, the cost of the naive algorithm is $\mathcal{O}(n^3)$.

Think a little bit: can Strassen algorithm help here?

- **Answer:** Yes. Applying the Strassen algorithm's core idea (which uses block matrix multiplication) to the recursive **Block LU-decomposition** leads to a theoretically faster algorithm with a complexity of $\mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.807})$.

Block LU-decomposition

We can try to compute the **block** version of LU-decomposition, splitting the matrix into sub-blocks:

$$\begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix} = \begin{pmatrix} \mathbf{L}_{11} & \mathbf{0} \\ \mathbf{L}_{21} & \mathbf{L}_{22} \end{pmatrix} \begin{pmatrix} \mathbf{U}_{11} & \mathbf{U}_{12} \\ \mathbf{0} & \mathbf{U}_{22} \end{pmatrix}$$

- There are two basic operations: compute LU-factorization of half-matrices + matrix-by-matrix product (which can be done recursively).

Example: Block-by-Block Multiplication

The matrix multiplication can be written in terms of the sub-blocks:

1. $\mathbf{A}_{11} = \mathbf{L}_{11} \mathbf{U}_{11}$
 2. $\mathbf{A}_{12} = \mathbf{L}_{11} \mathbf{U}_{12}$
 3. $\mathbf{A}_{21} = \mathbf{L}_{21} \mathbf{U}_{11}$
 4. $\mathbf{A}_{22} = \mathbf{L}_{21} \mathbf{U}_{12} + \mathbf{L}_{22} \mathbf{U}_{22}$
- **Comment:** The recursive computation of the **Schur complement** $(\mathbf{A}_{22} - \mathbf{L}_{21} \mathbf{U}_{12})$ is needed to find the decomposition of $\mathbf{A}_{22}$. This is how the Block LU-decomposition works: it requires one small LU-decomposition ($\mathbf{A}_{11}$), two forward/backward substitutions ($\mathbf{A}_{12}$, $\mathbf{A}_{21}$), and one Schur complement calculation ($\mathbf{A}_{22}$).

Existence of the LU-decomposition

- The LU-decomposition algorithm does not fail if we **do not divide by zero** at every step of the Gaussian elimination.

Q: when is it so, for which class of matrices?

A: It is true for **strictly regular matrices**.

Definition: Strictly Regular Matrices (Existence without Pivoting)

A square matrix $\mathbf{A}$ is called **strictly regular** (or sometimes **regular**) if and only if **all of its leading principal minors are non-zero**.

- **Principal Minor:** The determinant of the top-left square sub-matrix of size $k \times k$.
- **Condition:** $\det(\mathbf{A}[1 : k, 1 : k]) \neq 0$ for all $k = 1, \dots, n$.
- **Significance:** This is the formal mathematical condition guaranteeing that no division by zero (pivoting) will be necessary during Gaussian elimination, meaning the decomposition $\mathbf{A} = \mathbf{L}\mathbf{U}$ (without a permutation matrix $\mathbf{P}$) is guaranteed to exist and be unique (if the diagonal of $\mathbf{L}$ is fixed, e.g., to all ones).

Strictly regular matrices and LU decomposition

- **Definition:** A matrix $\mathbf{A}$ is called **strictly regular**, if all of its **leading principal minors** (i.e., submatrices in the first k rows and k columns) are non-singular ($\det \neq 0$).
- In this case, there always exists an LU-decomposition ($\mathbf{A} = \mathbf{L}\mathbf{U}$). The reverse is also true (check!).

The Reverse Check: Existence Implies Non-Singular Minors

The Statement to Check: If $\mathbf{A} = \mathbf{L}\mathbf{U}$ exists, then all leading principal minors of $\mathbf{A}$ are non-singular.

Proof Sketch for the Check:

1. Consider the $k \times k$ leading principal submatrix of $\mathbf{A}$, denoted $\mathbf{A}_k$.
2. The $\mathbf{L}\mathbf{U}$ multiplication on $\mathbf{A}$ is structured such that the $k \times k$ leading principal submatrix of $\mathbf{A}$ is equal to the product of the corresponding $k \times k$ leading principal submatrices of $\mathbf{L}$ and $\mathbf{U}$.

$$\mathbf{A}_k = \mathbf{L}_k \mathbf{U}_k$$

3. The determinant of a product is the product of determinants:

$$\det(\mathbf{A}_k) = \det(\mathbf{L}_k) \det(\mathbf{U}_k)$$

4. Since $\mathbf{L}$ and $\mathbf{U}$ are triangular, their determinants are the product of their diagonal elements.

$$\det(\mathbf{L}_k) = \prod_{i=1}^k l_{ii}, \quad \det(\mathbf{U}_k) = \prod_{i=1}^k u_{ii}$$

5. If the LU decomposition exists, $\mathbf{L}$ and $\mathbf{U}$ must have non-zero diagonals for the procedure to have been completed. Therefore, $\det(\mathbf{L}_k) \neq 0$ and $\det(\mathbf{U}_k) \neq 0$.
6. Thus, $\det(\mathbf{A}_k) \neq 0$, meaning all leading principal minors of $\mathbf{A}$ are non-singular. (The check passes.)

Corollary: Uniqueness of LU Decomposition (Unit Triangular)

Corollary: If $\mathbf{L}$ is unit triangular (ones on the diagonal), then LU-decomposition is **unique**.

Proof:

Indeed, $\mathbf{L}_1 \mathbf{U}_1 = \mathbf{L}_2 \mathbf{U}_2$. This means:

$$\mathbf{L}_2^{-1}\mathbf{L}_1 = \mathbf{U}_2\mathbf{U}_1^{-1}$$

- **LHS ($\mathbf{L}_2^{-1}\mathbf{L}_1$):** This is the product of two unit lower triangular matrices. The inverse of a unit lower triangular matrix is also unit lower triangular. The product is also **unit lower triangular**.
- **RHS ($\mathbf{U}_2\mathbf{U}_1^{-1}$):** This is the product of two upper triangular matrices. The product is also **upper triangular**.

The only matrix that is both unit lower triangular and upper triangular is the **Identity matrix (I)**.

$$\mathbf{L}_2^{-1}\mathbf{L}_1 = \mathbf{I} \quad \text{and} \quad \mathbf{U}_2\mathbf{U}_1^{-1} = \mathbf{I}$$

Thus, $\mathbf{L}_1 = \mathbf{L}_2$ and $\mathbf{U}_1 = \mathbf{U}_2$.

LU for positive definite Hermitian matrices (Cholesky factorization)

- Strictly regular matrices have LU-decomposition.
- An important subclass of strictly regular matrices is the class of **Hermitian positive definite matrices**.

Definition. A matrix $\mathbf{A}$ is called **positive definite** if for any $\mathbf{x} : \|\mathbf{x}\| \neq 0$ we have

$$(\mathbf{x}, \mathbf{Ax}) > 0.$$

- If this holds for $\mathbf{x} \in \mathbb{C}^n$, then the matrix $\mathbf{A}$ has to be Hermitian.
- If this holds for $\mathbf{x} \in \mathbb{R}^n$, then the matrix $\mathbf{A}$ can be non-symmetric.

Claim: A Hermitian positive definite matrix $\mathbf{A}$ is strictly regular and has **Cholesky factorization** of the form

$$\mathbf{A} = \mathbf{R}\mathbf{R}^*,$$

where $\mathbf{R}$ is a lower triangular matrix.

- *(Note: The existence of the factorization is guaranteed and is proven by showing that all leading principal minors of a positive definite matrix are positive, thus strictly regular.)*
- It is sometimes referred to as the **"square root"** of the matrix.

Example: Cholesky Factorization for a 2×2 Matrix

Find the Cholesky factorization $\mathbf{A} = \mathbf{R}\mathbf{R}^T$ (real, symmetric case) for:

$$\mathbf{A} = \begin{pmatrix} 4 & 6 \\ 6 & 13 \end{pmatrix}$$

Goal: $\begin{pmatrix} 4 & 6 \\ 6 & 13 \end{pmatrix} = \begin{pmatrix} r_{11} & 0 \\ r_{21} & r_{22} \end{pmatrix} \begin{pmatrix} r_{11} & r_{21} \\ 0 & r_{22} \end{pmatrix}$

Step-by-step Calculation (Matching elements):

1. **Element (1, 1):** $4 = r_{11}^2 + 0 \implies r_{11} = \sqrt{4} = 2$

2. **Element (2, 1) or (1, 2):** $6 = r_{21}r_{11} + 0 \implies 6 = r_{21}(2) \implies r_{21} = 3$

3. **Element (2, 2):** $13 = r_{21}^2 + r_{22}^2 \implies 13 = 3^2 + r_{22}^2 \implies 13 = 9 + r_{22}^2$

$$r_{22}^2 = 4 \implies r_{22} = 2$$

Result:

$$\mathbf{R} = \begin{pmatrix} 2 & 0 \\ 3 & 2 \end{pmatrix}$$

(Check: $\mathbf{R}\mathbf{R}^T = \begin{pmatrix} 2 & 0 \\ 3 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 6 & 9+4 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 6 & 13 \end{pmatrix} = \mathbf{A}$)

Computing LU-decomposition

- In many cases, computing LU-decomposition once is a good idea!
- Once the decomposition is found (it costs $\mathcal{O}(n^3)$ operations), then solving linear systems with $\mathbf{L}$ and $\mathbf{U}$ costs only $\mathcal{O}(n^2)$ operations.

Check:

- **Solving linear systems with triangular matrices is easy (why?).**
 - **Answer:** Because they can be solved directly using **forward substitution** ($\mathbf{L}\mathbf{y} = \mathbf{f}$) or **backward substitution** ($\mathbf{U}\mathbf{x} = \mathbf{y}$), which require only $\mathcal{O}(n^2)$ total operations.
- **How we compute the $\mathbf{L}$ and $\mathbf{U}$ factors?**
 - **Answer:** They are computed via Gaussian elimination by storing the multiplier at each elimination step into the lower triangular matrix $\mathbf{L}$.

When LU fails

- What happens, if the matrix is not strictly regular (or the **pivots** in the Gaussian elimination are really small)?
- There is a classical 2×2 example of a matrix with a bad LU decomposition.
- The matrix we look at is

$$\mathbf{A} = \begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix}$$

- If ϵ is sufficiently small, we **might fail** (numerically). In contrast the Cholesky factorization is **always stable**.
- **Is LU factorization forward or backward stable?**
 - **Answer:** Standard **LU without pivoting** is **not** generally stable (it can be unstable). **LU with partial pivoting** ($\mathbf{PA} = \mathbf{LU}$) is generally **backward stable**, which is why it is the standard algorithm.

Example: Demonstrating Instability (When ϵ is Small)

Let's find the LU decomposition of $\mathbf{A}$ and see the effect of a tiny ϵ .

1. LU Decomposition of $\mathbf{A} = \mathbf{LU}$:

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ l_{21} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{pmatrix}$$

- $u_{11} = \epsilon, u_{12} = 1.$
- $l_{21} = 1/\epsilon.$
- $u_{22} = 1 - l_{21}u_{12} = 1 - (1/\epsilon)(1) = \frac{\epsilon-1}{\epsilon}.$

Result:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} \epsilon & 1 \\ 0 & \frac{\epsilon-1}{\epsilon} \end{pmatrix}$$

2. Instability Analysis (Let $\epsilon = 10^{-20}$):

- **Large Multiplier:** $l_{21} = 10^{20}$. The lower triangular factor $\mathbf{L}$ contains an enormous number.
- **Contamination/Roundoff:** When solving $\mathbf{Ly} = \mathbf{f}$ and $\mathbf{Ux} = \mathbf{y}$, any small roundoff error in $\mathbf{f}$ or introduced during the $\mathbf{Ly}$ step will be multiplied by $1/\epsilon = 10^{20}$, completely **amplifying the error** and destroying the solution $\mathbf{x}$.

3. The Fix (Pivoting):

The correct way to solve this is to perform a row swap (**Pivoting**):

$$\mathbf{P}\mathbf{A} = \begin{pmatrix} 1 & 1 \\ \epsilon & 1 \end{pmatrix}$$

This new matrix is stable and gives an $\mathbf{L}$ that has no large elements, preventing error amplification.

The concept of pivoting

- We can do **pivoting**, i.e. permute rows and columns to maximize the pivot element $\mathbf{A}_{kk}$ that we divide over.
- The simplest but effective strategy is the **row pivoting** (or Partial Pivoting): at each step, select the index in the current column that is maximal in modulus, and put it onto the diagonal.
- It gives us the decomposition

$$\mathbf{A} = \mathbf{P}\mathbf{L}\mathbf{U},$$

where $\mathbf{P}$ is a **permutation matrix**.

Q. What makes row pivoting good?

A. It is made good by the fact that

$$|\mathbf{L}_{ij}| < 1,$$

but the elements of $\mathbf{U}$ can grow, up to 2^{n-1} (in practice, this is very rarely encountered).

- **Explanation:** By ensuring the pivot is the largest element in the column, the multiplier ($\mathbf{L}_{ij}$) is always ≤ 1 . This prevents the catastrophic error amplification seen when dividing by a very small pivot, making the factorization **backward stable**.
- **Can you come up with a matrix where the elements of $\mathbf{U}$ grow as much as possible?**

Example: Matrix Causing Maximum Growth in $\mathbf{U}$

The matrix that causes the worst-case growth in the $\mathbf{U}$ factor is a variation of the **Wilkinson matrix**, often demonstrated by the following $n \times n$ matrix (here for $n = 4$):

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & 1 \end{pmatrix}$$

Analysis of Growth:

- **The Problem:** The partial pivoting rule here is to *always* select the diagonal pivot (since it's $|1|$ and the elements below it are $| -1|$). However, Gaussian elimination using these pivots causes a massive fill-in and growth in the subsequent matrices.
- **The Result (for general n):** After the final step, the largest element in the resulting $\mathbf{U}$ matrix is $u_{n,n}$ (or $u_{1,n}$ in some variations).
- **Maximum Growth:** The value of the largest element in $\mathbf{U}$ can be proven to be 2^{n-1} .
- **Conclusion:** Although LU with Partial Pivoting is generally a reliable, stable algorithm, this class of matrices shows that the worst-case bounds for element growth are exponentially large, which is why it is only considered **backward stable** and not unconditionally stable.

Neumann series

- To study, why there can be such big errors in a solution (see the example above on the Hilbert matrix) we need an important auxiliary result.

Neumann series:

If a matrix $\mathbf{F}$ is such that $\|\mathbf{F}\| < 1$ holds, then the matrix $(\mathbf{I} - \mathbf{F})$ is invertible and

$$(\mathbf{I} - \mathbf{F})^{-1} = \mathbf{I} + \mathbf{F} + \mathbf{F}^2 + \mathbf{F}^3 + \dots = \sum_{k=0}^{\infty} \mathbf{F}^k.$$

Note that it is a matrix version of the geometric progression.

Q: Does this series converge if $\|\mathbf{F}\| > 1$ and $\|\mathbf{F}^2\| < 1$? What about $\|\mathbf{F}^k\| < 1$ for arbitrary k ?

A: No. Convergence of the series is guaranteed **if and only if** the entire series of norms converges, which is true if $\|\mathbf{F}\| < 1$. If $\|\mathbf{F}\| \geq 1$, convergence is not guaranteed, even if a single term like $\|\mathbf{F}^2\| < 1$.

Q: What norm is considered here? What is the "best possible" norm here?

A: Any **submultiplicative matrix norm** $\|\cdot\|$ can be used. The "best possible" convergence condition is based on the **spectral radius** $\rho(\mathbf{F})$.

Q: Is there a limit $\lim_{k \rightarrow \infty} \|\mathbf{F}^k\|^{\frac{1}{k}}$? How is it called?

A: Yes, the limit exists and is called the **spectral radius** $\rho(\mathbf{F})$. It is equal to the magnitude of the largest eigenvalue: $\rho(\mathbf{F}) = \max_i |\lambda_i(\mathbf{F})|$.

- The necessary and sufficient condition for convergence is $\rho(\mathbf{F}) < 1$.
-

Proof Sketch

The proof is constructive. First of all, prove that the series $\sum_{k=0}^{\infty} \mathbf{F}^k$ converges.

1. Proof by Telescoping Sum:

Like in the scalar case, we have the identity for a finite sum:

$$(\mathbf{I} - \mathbf{F}) \sum_{k=0}^N \mathbf{F}^k = (\mathbf{I} - \mathbf{F}^{N+1})$$

2. Showing Limit to Inverse:

We want to show that as $N \rightarrow \infty$, the right side goes to $\mathbf{I}$, thus proving the infinite sum is $(\mathbf{I} - \mathbf{F})^{-1}$.

$$(\mathbf{I} - \mathbf{F}) \sum_{k=0}^N \mathbf{F}^k = (\mathbf{I} - \mathbf{F}^{N+1}) \rightarrow \mathbf{I}, \quad N \rightarrow +\infty$$

3. Completing the Proof (Showing $\mathbf{F}^{N+1} \rightarrow \mathbf{0}$):

The required step is to show that $\mathbf{F}^{N+1} \rightarrow \mathbf{0}$ as $N \rightarrow \infty$. This is done by estimating the norm of the difference:

$$\begin{aligned} \|(\mathbf{I} - \mathbf{F}) \sum_{k=0}^N \mathbf{F}^k - \mathbf{I}\| &= \|(\mathbf{I} - \mathbf{F}^{N+1}) - \mathbf{I}\| \\ \|(\mathbf{I} - \mathbf{F}^{N+1}) - \mathbf{I}\| &= \|-\mathbf{F}^{N+1}\| = \|\mathbf{F}^{N+1}\| \end{aligned}$$

Indeed, using the submultiplicative property ($\|\mathbf{F}^{N+1}\| \leq \|\mathbf{F}\|^{N+1}$):

$$\|\mathbf{F}^{N+1}\| \leq \|\mathbf{F}\|^{N+1} \rightarrow 0, \quad \text{as } N \rightarrow +\infty$$

(Since $\|\mathbf{F}\| < 1$ by assumption, this geometric sequence of norms goes to zero, proving the convergence of the series to $(\mathbf{I} - \mathbf{F})^{-1}$).

Small perturbation of the inverse

- Using the Neumann series, we can estimate how the perturbation of the matrix influences the inverse matrix.
- We assume that the perturbation $\mathbf{E}$ is small in the sense that $\|\mathbf{A}^{-1}\mathbf{E}\| < 1$.
- Then, by letting $\mathbf{F} = -\mathbf{A}^{-1}\mathbf{E}$ in $(\mathbf{I} - \mathbf{F})^{-1} = \sum \mathbf{F}^k$:

$$(\mathbf{A} + \mathbf{E})^{-1} = (\mathbf{A}(\mathbf{I} - \mathbf{F}))^{-1} = (\mathbf{I} - \mathbf{F})^{-1} \mathbf{A}^{-1} = \sum_{k=0}^{\infty} (-\mathbf{A}^{-1} \mathbf{E})^k \mathbf{A}^{-1}$$

And moreover, the **relative error** of the inverse is bounded by:

$$\frac{\|(\mathbf{A} + \mathbf{E})^{-1} - \mathbf{A}^{-1}\|}{\|\mathbf{A}^{-1}\|} \leq \frac{\|\mathbf{A}^{-1}\| \|\mathbf{E}\|}{1 - \|\mathbf{A}^{-1} \mathbf{E}\|} \cdot \frac{\|\mathbf{I}\|}{\|\mathbf{A}^{-1}\|}$$

Comment: Since $\mathbf{E} \approx \Delta \mathbf{A}$, this bounds the change in the inverse based on the relative size of the perturbation.

As you see, the norm of the inverse enters the estimate.

Condition number of a linear system

Now consider the **perturbed linear system**:

$$(\mathbf{A} + \Delta \mathbf{A}) \hat{\mathbf{x}} = \mathbf{f} + \Delta \mathbf{f}.$$

The goal is to estimate the error in the solution, $\Delta \mathbf{x} = \hat{\mathbf{x}} - \mathbf{x}$, where $\mathbf{x} = \mathbf{A}^{-1} \mathbf{f}$.

Estimates (Derivation of Error Bound)

$$\hat{\mathbf{x}} - \mathbf{x} = (\mathbf{A} + \Delta \mathbf{A})^{-1} (\mathbf{f} + \Delta \mathbf{f}) - \mathbf{A}^{-1} \mathbf{f}$$

The derivation separates the error contribution from $\Delta \mathbf{A}$ and $\Delta \mathbf{f}$:

$$\hat{\mathbf{x}} - \mathbf{x} = [(\mathbf{A} + \Delta \mathbf{A})^{-1} - \mathbf{A}^{-1}] \mathbf{f} + (\mathbf{A} + \Delta \mathbf{A})^{-1} \Delta \mathbf{f}$$

Applying the Neumann series for $(\mathbf{A} + \Delta \mathbf{A})^{-1} - \mathbf{A}^{-1} = \mathbf{A}^{-1} \sum_{k=1}^{\infty} (-\Delta \mathbf{A} \mathbf{A}^{-1})^k$:

$$\hat{\mathbf{x}} - \mathbf{x} = \left[\sum_{k=1}^{\infty} (-\mathbf{A}^{-1} \Delta \mathbf{A})^k \mathbf{A}^{-1} \right] \mathbf{f} + \left[\sum_{k=0}^{\infty} (-\mathbf{A}^{-1} \Delta \mathbf{A})^k \mathbf{A}^{-1} \right] \Delta \mathbf{f}$$

Final Bound (The Significance)

Taking the norm and isolating the leading terms (assuming small perturbations), the solution error $\Delta \mathbf{x}$ is bounded by:

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim (\|\mathbf{A}\| \|\mathbf{A}^{-1}\|) \left(\frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\Delta \mathbf{f}\|}{\|\mathbf{f}\|} \right)$$

- **The Factor:** The term $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$ is the **Condition Number** of the matrix $\mathbf{A}$.

- **Conclusion:** The final bound shows that the *relative error* in the solution ($\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|}$) is proportional to the **Condition Number** multiplied by the *relative error* in the input data ($\frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\Delta \mathbf{f}\|}{\|\mathbf{f}\|}$).
- **Ill-Conditioned System:** If $\kappa(\mathbf{A})$ is large, a small error in the input data leads to a large error in the solution, meaning the system is **ill-conditioned**.
therefore

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{1}{\|\mathbf{A}^{-1}\mathbf{f}\|} \left[\frac{\|\mathbf{A}^{-1}\| \|\Delta \mathbf{A}\| \|\mathbf{f}\|}{1 - \|\mathbf{A}^{-1}\Delta \mathbf{A}\|} + \frac{\|\mathbf{A}^{-1}\| \|\Delta \mathbf{f}\|}{1 - \|\mathbf{A}^{-1}\Delta \mathbf{A}\|} \right]$$

Comment: The term $\|\mathbf{A}^{-1}\mathbf{f}\|$ is a lower bound for $\|\mathbf{x}\|$, which is used in the final step to simplify the denominator.

Note that $\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\| \|\mathbf{x}\|$, which means $\|\mathbf{f}\| \leq \|\mathbf{A}\| \|\mathbf{x}\|$. Re-arranging $\mathbf{x} = \mathbf{A}^{-1}\mathbf{f}$ gives the reciprocal estimate used here: $\|\mathbf{A}^{-1}\mathbf{f}\| \geq \frac{\|\mathbf{f}\|}{\|\mathbf{A}\|}$.

Now we are ready to get the final estimate:

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{\|\mathbf{A}\| \|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}^{-1}\Delta \mathbf{A}\|} \left(\frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\Delta \mathbf{f}\|}{\|\mathbf{f}\|} \right)$$

This is equivalent to:

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{\text{cond}(\mathbf{A})}{1 - \text{cond}(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|}} \left(\frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\Delta \mathbf{f}\|}{\|\mathbf{f}\|} \right)$$

The crucial role is played by the **condition number** $\text{cond}(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$.

Example: Ill-Conditioned System

The bound clearly shows that the **Condition Number** $\text{cond}(\mathbf{A})$ acts as an **amplification factor** for the relative error in the solution.

Scenario: We solve a linear system $\mathbf{A}\mathbf{x} = \mathbf{f}$ where:

- The input data has a small relative error: $\frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\Delta \mathbf{f}\|}{\|\mathbf{f}\|} \approx 10^{-10}$ (machine precision).
- The matrix is **ill-conditioned** with $\text{cond}(\mathbf{A}) = 10^5$.
- Assume the denominator term is close to 1 (i.e., $\text{cond}(\mathbf{A}) \frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|} \ll 1$).

Resulting Solution Error:

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \approx \mathbf{cond}(\mathbf{A}) \times (\text{Input Error})$$

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \approx 10^5 \times 10^{-10} = \mathbf{10^{-5}}$$

Conclusion: The initial 10-digit accuracy has been reduced to only 5-digit accuracy (10^{-5}) by the ill-conditioned nature of the matrix. This is the fundamental reason why solving systems with large condition numbers is numerically challenging.

Overdetermined linear systems

- An important class of problems are **overdetermined linear systems**, when the number of equations is greater than the number of unknowns.
- The simplest example that you all know, is **linear fitting**, fitting a set of 2D points by a line.
- Then, a typical way is to minimize the residual (least squares)

$$\|\mathbf{Ax} - \mathbf{b}\|_2 \rightarrow \min$$

Overdetermined system and Gram matrix

The optimality condition is $\mathbf{0} = \nabla (\|\mathbf{Ax} - \mathbf{b}\|_2^2)$, where ∇ denotes the gradient. Therefore,

$$\mathbf{0} = \nabla (\|\mathbf{Ax} - \mathbf{b}\|_2^2) = \nabla ((\mathbf{Ax} - \mathbf{b})^*(\mathbf{Ax} - \mathbf{b}))$$

$$\mathbf{0} = 2(\mathbf{A}^* \mathbf{Ax} - \mathbf{A}^* \mathbf{b}) = \mathbf{0}.$$

Thus, we arrive at the system of equations that defines the optimal solution $\mathbf{x}_{LS}$:

$$\mathbf{A}^* \mathbf{Ax} = \mathbf{A}^* \mathbf{b}$$

- The matrix $\mathbf{A}^* \mathbf{A}$ is called the **Gram matrix** and the system is called the **normal equation**.
- This is not a good way to do it, since the condition number of $\mathbf{A}^* \mathbf{A}$ is a square of the condition number of $\mathbf{A}$ (check why).

Check Why: Condition Number Squaring and Numerical Stability

The statement $\text{cond}(\mathbf{A}^* \mathbf{A}) = (\text{cond}(\mathbf{A}))^2$ is the reason why the Normal Equations are generally avoided in favor of methods like **QR Decomposition** or **SVD**.

Proof Sketch (using Spectral Norm $\|\cdot\|_2$):

1. **Recall SVD:** $\mathbf{A}$ has singular values $\sigma_1 \geq \dots \geq \sigma_n$.

$$\text{cond}_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$$

2. **Singular Values of Gram Matrix:** The singular values of $\mathbf{A}^* \mathbf{A}$ are σ_i^2 .
3. **Maximum/Minimum:** The max and min singular values of $\mathbf{A}^* \mathbf{A}$ are $\sigma_{\max}^2$ and $\sigma_{\min}^2$.
4. **Condition Number of Gram Matrix:**

$$\text{cond}_2(\mathbf{A}^* \mathbf{A}) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^2 = (\text{cond}_2(\mathbf{A}))^2$$

The Problem (Numerical Stability):

- If $\mathbf{A}$ has $\text{cond}_2(\mathbf{A}) = 10^8$ (a large but manageable number), then $\text{cond}_2(\mathbf{A}^* \mathbf{A}) = 10^{16}$.
- 10^{16} is the limit of working precision for standard 64-bit floating point numbers.
- Solving the normal equations $\mathbf{A}^* \mathbf{A} \mathbf{x} = \mathbf{A}^* \mathbf{b}$ effectively **squares the error amplification factor**, making the matrix $\mathbf{A}^* \mathbf{A}$ nearly singular and the solution extremely sensitive to roundoff error.

Example: The Linear Fitting Problem

Problem: Find the best fit line $y = x_1 + x_2 t$ for the following points (t_i, y_i) :

$$(1, 3), (2, 5), (3, 6), (4, 8)$$

This is an overdetermined system $\mathbf{A} \mathbf{x} = \mathbf{b}$ where $\mathbf{A}$ has 4 rows (equations) and 2 columns (unknowns x_1, x_2):

$$\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 6 \\ 8 \end{pmatrix}$$

Solving via Normal Equations:

1. **Form Gram Matrix ($\mathbf{A}^T \mathbf{A}$) and RHS ($\mathbf{A}^T \mathbf{b}$):**

$$\mathbf{A}^T \mathbf{A} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}$$

$$\mathbf{A}^T \mathbf{b} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 5 \\ 6 \\ 8 \end{pmatrix} = \begin{pmatrix} 22 \\ 65 \end{pmatrix}$$

2. Solve Normal Equation $\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$:

$$\begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 22 \\ 65 \end{pmatrix}$$

The solution is $\mathbf{x} = \begin{pmatrix} 1.5 \\ 1.6 \end{pmatrix}$.

(Note: This solution is numerically exact for this simple example, but for a large, poorly conditioned $\mathbf{A}$, this method would be unstable.)

Pseudoinverse

- Matrix $\mathbf{A}^* \mathbf{A}$ can be **singular** in the general case (when $\mathbf{A}$ does not have full column rank).
- Therefore, we need to introduce the concept of **pseudoinverse matrix** $\mathbf{A}^\dagger$ such that the solution of the linear least squares problem can formally be written as

$$\mathbf{x} = \mathbf{A}^\dagger \mathbf{b}.$$

- The matrix

$$\mathbf{A}^\dagger = \lim_{\alpha \rightarrow 0} (\alpha \mathbf{I} + \mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^*$$

is called the **Moore–Penrose pseudoinverse** of the matrix $\mathbf{A}$.

- If matrix $\mathbf{A}$ has **full column rank**, then $\mathbf{A}^* \mathbf{A}$ is non-singular and we get the classic least squares solution:

$$\mathbf{A}^\dagger = \lim_{\alpha \rightarrow 0} (\alpha \mathbf{I} + \mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* = (\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^*$$

- If matrix $\mathbf{A}$ is squared and non-singular, the pseudoinverse reduces to the standard inverse of $\mathbf{A}$:

$$\mathbf{A}^\dagger = \lim_{\alpha \rightarrow 0} (\alpha \mathbf{I} + \mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* = (\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* = \mathbf{A}^{-1}$$

- If $\mathbf{A}$ has **linearly dependent columns** (or $\mathbf{A}^* \mathbf{A}$ is singular), then $\mathbf{A}^\dagger \mathbf{b}$ gives the solution that has **minimal Euclidean norm** ($\|\mathbf{x}\|_2$).

The Computational Core: Pseudoinverse via SVD

In practice, the most robust way to compute the pseudoinverse is via the **Singular Value Decomposition (SVD)**, as it avoids inverting the potentially singular Gram matrix entirely.

Let the SVD of $\mathbf{A}$ be $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^*$. The pseudoinverse $\mathbf{A}^\dagger$ is defined by:

$$\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^\dagger \mathbf{U}^*$$

where $\mathbf{\Sigma}^\dagger$ is constructed by:

1. Taking the reciprocal of every non-zero singular value σ_i (i.e., replacing σ_i with $1/\sigma_i$).
2. Transposing the resulting matrix.

Example: Pseudoinverse of a Rank-Deficient Matrix

Consider the rank-deficient matrix $\mathbf{A}$ from a previous example:

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

1. **SVD of $\mathbf{A}$ (from previous derivation):**

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Singular Values: $\sigma_1 = \sqrt{2}$, $\sigma_2 = 0$.

2. **Construct $\mathbf{\Sigma}^\dagger$:**

- $\sigma_1 \rightarrow 1/\sqrt{2}$
- $\sigma_2 \rightarrow 0$ (reciprocal of zero is zero).

$$\mathbf{\Sigma}^\dagger = \begin{pmatrix} 1/\sqrt{2} & 0 \\ 0 & 0 \end{pmatrix}$$

3. **Compute $\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^\dagger \mathbf{U}^T$:**

$$\mathbf{A}^\dagger = \left(\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \right) \begin{pmatrix} 1/\sqrt{2} & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

$$\mathbf{A}^\dagger = \begin{pmatrix} 1/2 & 0 \\ 1/2 & 0 \end{pmatrix}$$

Conclusion: The pseudoinverse $\mathbf{A}^\dagger$ allows the least-squares problem $\mathbf{A}\mathbf{x} \approx \mathbf{b}$ to be solved even when the normal equations $\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$ fail due to singularity. The resulting solution $\mathbf{x} = \mathbf{A}^\dagger \mathbf{b}$ is the unique one that minimizes the residual $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ and has the minimal vector norm $\|\mathbf{x}\|_2$.

A canonical way to solve linear least squares

The canonical and most numerically stable way to solve the least squares problem $\min \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ is to use the **QR decomposition**.

- Any matrix $\mathbf{A}$ can be factored into a product

$$\mathbf{A} = \mathbf{Q}\mathbf{R},$$

where $\mathbf{Q}$ is a **unitary** matrix (orthogonal in the real case), and $\mathbf{R}$ is an **upper triangular** matrix.

- Then, if $\mathbf{A}$ has **full column rank**, the least squares solution $\mathbf{x}_{LS}$ is found by substituting $\mathbf{A} = \mathbf{Q}\mathbf{R}$ into the Normal Equations solution $\mathbf{x} = (\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* \mathbf{b}$:

$$\mathbf{x} = (\mathbf{A}^* \mathbf{A})^{-1} \mathbf{A}^* \mathbf{b} = ((\mathbf{Q}\mathbf{R})^* (\mathbf{Q}\mathbf{R}))^{-1} (\mathbf{Q}\mathbf{R})^* \mathbf{b}$$

$$\mathbf{x} = (\mathbf{R}^* \mathbf{Q}^* \mathbf{Q} \mathbf{R})^{-1} \mathbf{R}^* \mathbf{Q}^* \mathbf{b}$$

Since $\mathbf{Q}$ is unitary, $\mathbf{Q}^* \mathbf{Q} = \mathbf{I}$ (Identity Matrix).

$$\mathbf{x} = (\mathbf{R}^* \mathbf{R})^{-1} \mathbf{R}^* \mathbf{Q}^* \mathbf{b}$$

$$\mathbf{x} = \mathbf{R}^{-1} \mathbf{R}^* \mathbf{R}^{-1} \mathbf{R}^* \mathbf{Q}^* \mathbf{b} = \mathbf{R}^{-1} \mathbf{Q}^* \mathbf{b}$$

- Thus, finding the optimal solution $\mathbf{x}$ is equivalent to solving the upper triangular system:

$$\mathbf{R}\mathbf{x} = \mathbf{Q}^* \mathbf{b}$$

- Since $\mathbf{R}$ is upper triangular, the solving of this linear system costs only $\mathcal{O}(n^2)$ (using backward substitution).
- Also it is **more stable** than using the pseudoinverse matrix directly or solving the Normal Equations.

Summary of Numerical Stability (Why QR is Canonical)

The transformation from $\min \|\mathbf{Ax} - \mathbf{b}\|_2$ to $\mathbf{Rx} = \mathbf{Q}^*\mathbf{b}$ is numerically superior because:

1. **Avoids Condition Number Squaring:** The QR method works directly with $\mathbf{A}$ (Condition Number $\text{cond}(\mathbf{A})$), whereas the Normal Equations work with $\mathbf{A}^*\mathbf{A}$ (Condition Number $\text{cond}(\mathbf{A})^2$). The QR method thus avoids squaring the amplification of error.
2. **Unitary Transformation:** The multiplication $\mathbf{Q}^*\mathbf{b}$ involves a unitary matrix $\mathbf{Q}$, which is **perfectly stable** as it does not change the norm (length) of the vector, thus preventing any amplification of the error in $\mathbf{b}$.
3. **Efficiency:** The expensive $\mathcal{O}(n^3)$ part is the QR decomposition itself, but the final step to solve for $\mathbf{x}$ is fast ($\mathcal{O}(n^2)$ back-substitution).

What is an eigenvector?

- **Definition.** A vector $\mathbf{x} \neq \mathbf{0}$ is called an **eigenvector** of a square matrix $\mathbf{A}$ if there exists a number λ such that

$$\mathbf{Ax} = \lambda\mathbf{x}.$$

- The number λ is called an **eigenvalue**. The name **eigenpair** is also used.
- Since $\mathbf{A} - \lambda\mathbf{I}$ should have a non-trivial kernel (nullspace), eigenvalues are the roots of the characteristic polynomial:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0.$$

Eigendecomposition

If matrix $\mathbf{A}$ of size $n \times n$ has n linearly independent eigenvectors $\mathbf{s}_i, i = 1, \dots, n$:

$$\mathbf{As}_i = \lambda_i\mathbf{s}_i,$$

then this can be written as a single matrix equation:

$$\mathbf{AS} = \mathbf{S}\mathbf{\Lambda},$$

where $\mathbf{S} = (\mathbf{s}_1, \dots, \mathbf{s}_n)$ (the matrix of eigenvectors), $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ (the diagonal matrix of eigenvalues).

Or equivalently:

$$\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1}.$$

- This is called **eigendecomposition** of a matrix. Matrices that can be represented by their eigendecomposition are called **diagonalizable**.

Existence (Answered)

- **What classes of matrices are diagonalizable?**
 - **Answer:** Matrices with **distinct eigenvalues** are always diagonalizable.
 - **Also diagonalizable:** All **Hermitian** (symmetric in the real case) matrices are guaranteed to be diagonalizable (they have a complete set of orthonormal eigenvectors).
 - **Simple example can be matrices with all different eigenvalues.**
 - **Answer:** Yes, if all n eigenvalues are distinct, the n corresponding eigenvectors are guaranteed to be linearly independent, making the matrix diagonalizable.
-

Example: Eigendecomposition of a 2×2 Matrix

Let $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$.

1. Find Eigenvalues (λ):

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \begin{vmatrix} 3 - \lambda & 1 \\ 0 & 2 - \lambda \end{vmatrix} = (3 - \lambda)(2 - \lambda) - 0 = 0$$

The eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 2$. (Since they are distinct, $\mathbf{A}$ is diagonalizable).

$$\mathbf{\Lambda} = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}$$

2. Find Eigenvectors ($\mathbf{s}$):

- **For $\lambda_1 = 3$:** Solve $(\mathbf{A} - 3\mathbf{I})\mathbf{s}_1 = \mathbf{0}$.

$$\begin{pmatrix} 0 & 1 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} s_{11} \\ s_{21} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies s_{21} = 0 \implies \mathbf{s}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

- **For $\lambda_2 = 2$:** Solve $(\mathbf{A} - 2\mathbf{I})\mathbf{s}_2 = \mathbf{0}$.

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} s_{12} \\ s_{22} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies s_{12} + s_{22} = 0 \implies \mathbf{s}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

3. The Eigendecomposition:

$$\mathbf{S} = \begin{pmatrix} 1 & 1 \\ 0 & -1 \end{pmatrix}$$

The decomposition is:

$$\mathbf{A} = \mathbf{S}\mathbf{\Lambda}\mathbf{S}^{-1} = \begin{pmatrix} 1 & 1 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & -1 \end{pmatrix}^{-1}$$

Properties of determinant

Determinant has many nice properties:

1. $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$ (Multiplicativity)
 2. If we have one row as a sum of two vectors, determinant is a sum of two determinants (Linearity in one row/column).
 3. "Minor expansion": we can expand determinant through a selected row or column (Laplace's formula).
- If you do it via **minor expansion**, we get **exponential** complexity in n ($\mathcal{O}(n!)$).
 - **Can we do $\mathcal{O}(n^3)$?**
 - **Answer:** Yes. The determinant can be computed in $\mathcal{O}(n^3)$ time by using **Gaussian Elimination (LU Decomposition)**. Since the determinant of a triangular matrix is the product of its diagonal elements, $\det(\mathbf{A}) = \det(\mathbf{L}) \det(\mathbf{U}) = \prod_{i=1}^n u_{ii}$ (assuming $\mathbf{L}$ is unit triangular).

Eigenvalues and characteristic equation

- Now we go back to the eigenvalues.
- The characteristic equation can be used to compute the eigenvalues, which leads to the **naïve algorithm**:

$$p(\lambda) = \det(\mathbf{A} - \lambda \mathbf{I})$$

1. Compute coefficients of the polynomial
2. Compute the roots

Is this a good idea?

Give your feedback:

- **Answer (Feedback):** No, this is a very bad idea for large matrices.

Why the Naïve Algorithm Fails (Feedback)

1. **Cost (Step 1):** Computing the coefficients of $p(\lambda)$ requires n separate determinant computations, each of which costs $\mathcal{O}(n^3)$. This makes Step 1 cost $\mathcal{O}(n^4)$ or more, which is already too slow for large n .
2. **Accuracy (Step 2):** Finding the roots of a high-degree polynomial is an **extremely ill-conditioned** problem. Even for relatively small n (like $n = 20$), small errors in the polynomial coefficients can lead to huge errors in the calculated roots (eigenvalues). The slightest numerical error in Step 1 will destroy the accuracy of the eigenvalues in Step 2.
- **Modern Solution:** Modern numerical methods (like the **QR algorithm**) do not use the characteristic polynomial. They compute the eigenvalues of $\mathbf{A}$ directly via iterative transformations, which is significantly faster ($\mathcal{O}(n^3)$ to $\mathcal{O}(n^4)$) and much more numerically stable.

Gershgorin circles

- There is a very interesting theorem that sometimes helps to localize the eigenvalues.
- It is called the **Gershgorin theorem** (or the **Gershgorin Circle Theorem**).
- It states that all eigenvalues $\lambda_i, i = 1, \dots, n$ are located inside the union of **Gershgorin circles** C_i , where C_i is a disk on the complex plane with center a_{ii} and radius r_i defined by the sum of the absolute values of the off-diagonal elements in the i -th row:

$$r_i = \sum_{j \neq i} |a_{ij}|.$$

- Moreover, if the circles do not intersect, they contain only one eigenvalue per circle.
- The proof is instructive since it uses the concepts we looked at in the previous lectures (Neumann Series, non-singularity).

Proof Sketch

First, we need to show that if the matrix $\mathbf{A}$ is **strictly diagonally dominant** (SDD), i.e.,

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|,$$

then such a matrix is **non-singular** ($\det(\mathbf{A}) \neq 0$).

1. Non-Singularity via Neumann Series:

We separate the diagonal part ($\mathbf{D}$) and off-diagonal part ($\mathbf{S}$) of $\mathbf{A}$:

$$\mathbf{A} = \mathbf{D} + \mathbf{S} = \mathbf{D}(\mathbf{I} + \mathbf{D}^{-1}\mathbf{S}),$$

where $\mathbf{D}^{-1}\mathbf{S}$ is the matrix used for the radius calculation (scaled off-diagonal part).

The SDD condition ensures that the maximum row sum norm $\|\mathbf{D}^{-1}\mathbf{S}\|_{\infty} < 1$.

Therefore, by using the **Neumann series**, the matrix $\mathbf{I} + \mathbf{D}^{-1}\mathbf{S}$ is invertible, and since $\mathbf{D}$ is diagonal and invertible (due to the SDD condition), **hence $\mathbf{A}$ is invertible** (non-singular).

2. Proof by Contradiction (Linking SDD to Eigenvalues):

Now the proof of the full theorem follows by contradiction:

- Assume λ is an eigenvalue that **lies outside** of all Gershgorin circles.
- This means that for the matrix $\mathbf{B} = \mathbf{A} - \lambda\mathbf{I}$, the main diagonal entries $b_{ii} = a_{ii} - \lambda$ satisfy the SDD condition: $|a_{ii} - \lambda| > r_i$.
- Thus, the matrix $\mathbf{A} - \lambda\mathbf{I}$ is **strictly diagonally dominant** (SDD).
- From Step 1, this means $\mathbf{A} - \lambda\mathbf{I}$ is **invertible** (non-singular).
- But if λ is an eigenvalue, $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ has a non-zero solution, which means $(\mathbf{A} - \lambda\mathbf{I})\mathbf{x} = \mathbf{0}$ means $\mathbf{x} = \mathbf{0}$, which contradicts the assumption that $\mathbf{x} \neq \mathbf{0}$.
- **Contradiction:** Therefore, the eigenvalue λ **must** lie inside the union of the circles.

Example: Calculating and Using Gershgorin Circles

Consider the matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{pmatrix} 10 & -1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 5 \end{pmatrix}$$

We will define three Gershgorin circles C_1, C_2, C_3 :

Circle (C_i)	Center (a_{ii})	Radius (r_i)	Interpretation
C_1	$a_{11} = 10$	$r_1 = 1$	-1
C_2	$a_{22} = 1$	$r_2 = 1$	1
C_3	$a_{33} = 5$	$r_3 = 1$	0

Conclusion:

1. **Localization:** All three eigenvalues of $\mathbf{A}$ must lie within the union of the intervals $[-1, 3] \cup [4, 6] \cup [9, 11]$ on the real axis (since this real matrix has real eigenvalues).
2. **Isolation:** Since the circles C_1 (center 10, radius 1) and C_3 (center 5, radius 1) **do not intersect** with any other circle, the theorem guarantees that **each of these non-**

intersecting circles contains exactly one eigenvalue.

There is exactly one eigenvalue in $[9, 11]$.

There is exactly one eigenvalue in $[4, 6]$.

The final eigenvalue must be in $[-1, 3]$.

(The actual eigenvalues are approximately 10.038, 5.060, and 0.899).*

Power method

- The eigenvalue problem can be rewritten as a **fixed-point iteration**:

$$\mathbf{Ax} = \lambda \mathbf{x}, \quad \|\mathbf{x}\|_2 = 1 \quad \text{for stability.}$$

- This iteration is called the **power method** and finds the **largest in modulus eigenvalue** (λ_1) of $\mathbf{A}$.

Power method has the form:

- Initial iteration (un-normalized):

$$\mathbf{x}_{k+1} = \mathbf{Ax}_k$$

- Normalization step (for stability, keeping $\|\mathbf{x}_{k+1}\|_2 = 1$):

$$\mathbf{x}_{k+1} := \frac{\mathbf{x}_{k+1}}{\|\mathbf{x}_{k+1}\|_2}$$

- Convergence:** The iteration $\mathbf{x}_{k+1} \rightarrow \mathbf{v}_1$, where $\mathbf{Av}_1 = \lambda_1 \mathbf{v}_1$ and λ_1 is the largest eigenvalue in modulus and $\mathbf{v}_1$ is the corresponding eigenvector.
- On the $(k+1)$ -th iteration, the approximation to λ_1 can be found using the **Rayleigh Quotient** (for Hermitian matrices):

$$\lambda^{(k+1)} = (\mathbf{Ax}_{k+1}, \mathbf{x}_{k+1})$$

Comment: This is the scalar product of the current vector and the next vector.

- Note that $\lambda^{(k+1)}$ is not required for the $(k+2)$ -th iteration, but might be useful to measure error on each iteration: $\|\mathbf{Ax}_{k+1} - \lambda^{(k+1)} \mathbf{x}_{k+1}\|$.
- The convergence is geometric, but the **convergence ratio** is $q = \left| \frac{\lambda_2}{\lambda_1} \right| < 1$, for $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, and k is the number of iterations.
- It means the convergence can be arbitrarily small (slow). To prove it, it is sufficient to consider a 2×2 diagonal matrix.

Example: Convergence Rate for a Diagonal Matrix

Let $\mathbf{A}$ be a diagonal matrix, where the eigenvalues are the diagonal elements:

$$\mathbf{A} = \begin{pmatrix} 5 & 0 \\ 0 & 4 \end{pmatrix}$$

1. Determine Eigenvalues and Eigenvectors:

- $\lambda_1 = 5$ (Largest in modulus) with $\mathbf{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
- $\lambda_2 = 4$ with $\mathbf{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

2. Determine Convergence Ratio (q):

The convergence rate is governed by the ratio of the second largest to the largest eigenvalue.

$$q = \left| \frac{\lambda_2}{\lambda_1} \right| = \left| \frac{4}{5} \right| = \mathbf{0.8}$$

3. Interpretation of Convergence:

- **Fast/Slow:** Since $q = 0.8$ is close to 1, the convergence will be relatively **slow**. Each step reduces the error by a factor of 0.8.
- **Example Iteration:** If the initial error (in the direction of $\mathbf{v}_1$) is E_0 , the error after 10 iterations will be $E_{10} \approx E_0 \cdot (0.8)^{10} \approx E_0 \cdot 0.107$. The method is slow to achieve high precision.

4. The Slowest Convergence Case (Conceptual):

If the matrix was $\mathbf{A} = \begin{pmatrix} 5 & 0 \\ 0 & 4.9 \end{pmatrix}$, then the convergence ratio would be $q = \frac{4.9}{5.0} = 0.98$, and the method would converge **extremely slowly** (it would take 50 times more iterations to achieve the same precision). This demonstrates why the power method is only practical when $|\lambda_2| \ll |\lambda_1|$.

Power method

- The eigenvalue problem can be rewritten as a **fixed-point iteration**:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}, \quad \|\mathbf{x}\|_2 = 1 \quad \text{for stability.}$$

- This iteration is called the **power method** and finds the **largest in modulus eigenvalue** (λ_1) of $\mathbf{A}$.

Power method has the form:

- Initial iteration (un-normalized):

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$$

- Normalization step (for stability, keeping $\|\mathbf{x}_{k+1}\|_2 = 1$):

$$\mathbf{x}_{k+1} := \frac{\mathbf{x}_{k+1}}{\|\mathbf{x}_{k+1}\|_2}$$

- **Convergence:** The iteration $\mathbf{x}_{k+1} \rightarrow \mathbf{v}_1$, where $\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1$ and λ_1 is the largest eigenvalue in modulus and $\mathbf{v}_1$ is the corresponding eigenvector.
- On the $(k+1)$ -th iteration, the approximation to λ_1 can be found using the **Rayleigh Quotient** (for Hermitian matrices):

$$\lambda^{(k+1)} = (\mathbf{A}\mathbf{x}_{k+1}, \mathbf{x}_{k+1})$$

Comment: This is the scalar product of the current vector and the next vector.

- Note that $\lambda^{(k+1)}$ is not required for the $(k+2)$ -th iteration, but might be useful to measure error on each iteration: $\|\mathbf{A}\mathbf{x}_{k+1} - \lambda^{(k+1)}\mathbf{x}_{k+1}\|$.
- The convergence is geometric, but the **convergence ratio** is $q = \left|\frac{\lambda_2}{\lambda_1}\right| < 1$, for $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$, and k is the number of iterations.
- It means the convergence can be arbitrarily small (slow). To prove it, it is sufficient to consider a 2×2 diagonal matrix.

Example: Convergence Rate for a Diagonal Matrix

Let $\mathbf{A}$ be a diagonal matrix, where the eigenvalues are the diagonal elements:

$$\mathbf{A} = \begin{pmatrix} 5 & 0 \\ 0 & 4 \end{pmatrix}$$

1. Determine Eigenvalues and Eigenvectors:

- $\lambda_1 = 5$ (Largest in modulus) with $\mathbf{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
- $\lambda_2 = 4$ with $\mathbf{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$

2. Determine Convergence Ratio (q):

The convergence rate is governed by the ratio of the second largest to the largest eigenvalue.

$$q = \left|\frac{\lambda_2}{\lambda_1}\right| = \left|\frac{4}{5}\right| = 0.8$$

3. Interpretation of Convergence:

- **Fast/Slow:** Since $q = 0.8$ is close to 1, the convergence will be relatively **slow**. Each step reduces the error by a factor of 0.8.
- **Example Iteration:** If the initial error (in the direction of $\mathbf{v}_1$) is E_0 , the error after 10 iterations will be $E_{10} \approx E_0 \cdot (0.8)^{10} \approx E_0 \cdot 0.107$. The method is slow to achieve high precision.

4. The Slowest Convergence Case (Conceptual):

If the matrix was $\mathbf{A} = \begin{pmatrix} 5 & 0 \\ 0 & 4.9 \end{pmatrix}$, then the convergence ratio would be $q = \frac{4.9}{5.0} = 0.98$, and the method would converge **extremely slowly** (it would take 50 times more iterations to achieve the same precision). This demonstrates why the power method is only practical when $|\lambda_2| \ll |\lambda_1|$.

Things to remember about the power method

- **Result:** Power method gives an estimate of the **largest eigenvalue in modulus** (or **spectral radius**) of the given matrix.
- **Cost Efficiency (Tractability):** One step requires one matrix-by-vector product. If the matrix allows for an $\mathcal{O}(n)$ matvec (for example, it is **sparse** or structured), then the power method is **tractable** for larger n .
 - *Comment: For a dense matrix, one step is $\mathcal{O}(n^2)$, which is slower than sparse methods but still much faster than computing all eigenvalues via $\mathcal{O}(n^3)$ direct methods.*
- **Convergence Rate:** Convergence can be **slow** (governed by $|\lambda_2/\lambda_1|$).
- **Utility:** If only a **rough estimate** is needed, only a few iterations are sufficient.
- **Subspace:** The solution vector $\mathbf{x}_k$ is in the **Krylov subspace** $\mathcal{K}_k = \text{span}\{\mathbf{x}_0, \mathbf{A}\mathbf{x}_0, \dots, \mathbf{A}^k\mathbf{x}_0\}$ and has the form $\mu\mathbf{A}^k\mathbf{x}_0$, where μ is the normalization constant.
 - *Comment: The Krylov subspace is the space generated by repeatedly multiplying the initial vector by the matrix. Methods like Arnoldi/Lanczos exploit this subspace to find eigenvalues more efficiently than the basic Power Method.*

Matrix decomposition: the Schur form

There is one class of matrices when eigenvalues can be found easily: **triangular matrices**

$$\mathbf{A} = \begin{pmatrix} \lambda_1 & * & * \\ 0 & \lambda_2 & * \\ 0 & 0 & \lambda_3 \end{pmatrix}.$$

The eigenvalues of $\mathbf{A}$ are $\lambda_1, \lambda_2, \lambda_3$. **Why?**

Because the characteristic polynomial simplifies directly:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (\lambda_1 - \lambda)(\lambda_2 - \lambda)(\lambda_3 - \lambda) = 0.$$

- **Comment (Significance of Triangular Form):** Since the determinant of a triangular matrix is the product of its diagonal entries, the eigenvalues are simply the diagonal entries. Thus, if we can transform any matrix into a triangular form, we can immediately read off its eigenvalues.
- Thus, computing the eigenvalues of triangular matrices is easy. Now, the **unitary matrices** come to help.
- Let $\mathbf{U}$ be a unitary matrix, i.e., $\mathbf{U}^* \mathbf{U} = \mathbf{I}$. Then we prove that the transformed matrix $\mathbf{U}^* \mathbf{A} \mathbf{U}$ has the same eigenvalues as $\mathbf{A}$ (they are **similar**):

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det(\mathbf{U}(\mathbf{U}^* \mathbf{A} \mathbf{U} - \lambda \mathbf{I})\mathbf{U}^*)$$

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det(\mathbf{U} \mathbf{U}^*) \det(\mathbf{U}^* \mathbf{A} \mathbf{U} - \lambda \mathbf{I})$$

$$\det(\mathbf{A} - \lambda \mathbf{I}) = \det(\mathbf{U}^* \mathbf{A} \mathbf{U} - \lambda \mathbf{I}).$$

Comment: Since $\mathbf{U} \mathbf{U}^ = \mathbf{I}$ and $\det(\mathbf{I}) = 1$, the determinants (and thus the eigenvalues) are identical.**

- It means that the matrices $\mathbf{U}^* \mathbf{A} \mathbf{U}$ and $\mathbf{A}$ have the same characteristic polynomials, and the **same eigenvalues**.
- If we manage to make $\mathbf{U}^* \mathbf{A} \mathbf{U} = \mathbf{T}$ where $\mathbf{T}$ is **upper triangular**, then we are done (we can read the eigenvalues from $\mathbf{T}$).
- Multiplying from the left and the right by $\mathbf{U}$ and $\mathbf{U}^*$ respectively, we get the desired decomposition:

$$\mathbf{A} = \mathbf{U} \mathbf{T} \mathbf{U}^*.$$

This is the celebrated **Schur decomposition**.

- **Recall that unitary matrices imply stability** (they do not amplify errors), thus the eigenvalues are computed very accurately.
- The Schur decomposition shows why we need matrix decompositions: it represents a matrix into a product of three matrices with a simple structure, allowing eigenvalues to be found stably.

Schur theorem

Theorem: Every $\mathbf{A} \in \mathbb{C}^{n \times n}$ matrix can be represented in the **Schur form** $\mathbf{A} = \mathbf{U}\mathbf{T}\mathbf{U}^*$, where $\mathbf{U}$ is unitary and $\mathbf{T}$ is upper triangular.

Sketch of the proof (Recursive)

1. **Find the first Eigenpair:** Every matrix has at least one non-zero eigenvector $\mathbf{v}_1$ (from the Fundamental Theorem of Algebra on the characteristic polynomial).

$$\mathbf{A}\mathbf{v}_1 = \lambda_1\mathbf{v}_1, \quad \|\mathbf{v}_1\|_2 = 1$$

2. **Reduction Step:** Let $\mathbf{U}_1 = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n]$, where $\mathbf{v}_2, \dots, \mathbf{v}_n$ are any vectors orthogonal to $\mathbf{v}_1$, completing the unitary basis (e.g., using Householder reflections or Gram-Schmidt).

The unitary transformation $\mathbf{U}_1^*\mathbf{A}\mathbf{U}_1$ gives the **block triangular form**:

$$\mathbf{U}_1^*\mathbf{A}\mathbf{U}_1 = \begin{pmatrix} \lambda_1 & * \\ \mathbf{0} & \mathbf{A}_2 \end{pmatrix},$$

where $\mathbf{A}_2$ is an $(n-1) \times (n-1)$ matrix.

- This is called **block triangular form**. We can now work with $\mathbf{A}_2$ only and so on.
- **Note:** Since we need an eigenvector $\mathbf{v}_1$ in this proof, this proof is not a practical algorithm (it's hard to find eigenvectors accurately). The practical algorithm (the QR algorithm) finds the Schur form iteratively without ever explicitly calculating an eigenvector.

Application of the Schur theorem

- An important application of the Schur theorem: the analysis of **normal matrices**.

Definition. Matrix $\mathbf{A}$ is called a **normal matrix**, if its commute property holds:

$$\mathbf{A}\mathbf{A}^* = \mathbf{A}^*\mathbf{A}.$$

Q: Examples of normal matrices?

A: Examples: **Hermitian matrices** ($\mathbf{A} = \mathbf{A}^*$), **unitary matrices** ($\mathbf{A}\mathbf{A}^* = \mathbf{A}^*\mathbf{A} = \mathbf{I}$), and **skew-Hermitian matrices** ($\mathbf{A} = -\mathbf{A}^*$).

- **The Special Property:** If a matrix is **normal**, its Schur form $\mathbf{T}$ is not just upper triangular, it is **diagonal** ($\mathbf{T} = \mathbf{\Lambda}$). This means normal matrices are exactly the set of matrices that are diagonalizable by a unitary matrix.

Normal matrices

Theorem: A is a normal matrix ($AA^* = A^*A$), if and only if (iff) $A = U\Lambda U^*$, where U is unitary and Λ is diagonal.

Proof (Both Directions)

Part 1: Forward Direction (Diagonalizable $\implies$ Normal)

Statement: If $A = U\Lambda U^*$, where U is unitary and Λ is diagonal, then A is normal.

Proof:

We need to show $AA^* = A^*A$.

1. Calculate A^* :

$$A^* = (U\Lambda U^*)^* = (U^*)^* \Lambda^* U^* = U \Lambda^* U^*$$

2. Calculate AA^* :

$$AA^* = (U\Lambda U^*)(U\Lambda^* U^*) = U\Lambda(U^*U)\Lambda^* U^*$$

Since U is unitary, $U^*U = I$: $AA^* = U(\Lambda\Lambda^*)U^*$

3. Calculate A^*A :

$$A^*A = (U\Lambda^* U^*)(U\Lambda U^*) = U\Lambda^*(U^*U)\Lambda U^*$$

$$A^*A = U(\Lambda^*\Lambda)U^*$$

4. **Conclusion:** Since Λ is a diagonal matrix, $\Lambda\Lambda^* = \Lambda^*\Lambda$ (the order of multiplication does not matter for diagonal matrices). Therefore, $AA^* = A^*A$, and A is normal.

Part 2: Reverse Direction (Normal $\implies$ Diagonalizable)

Statement: If A is normal, then it is unitarily diagonalizable ($A = U\Lambda U^*$).

Proof:

We use the **Schur Decomposition Theorem**, which guarantees that every matrix A can be written as $A = UTU^*$, where T is upper triangular.

1. **Assume the Schur Form and Normality:**

Assume $A = UTU^*$ and $AA^* = A^*A$.

2. **Translate Normality to T :**

$$AA^* = A^*A \implies (UTU^*)(UTU^*)^* = (UTU^*)^*(UTU^*)$$

$$\mathbf{U}\mathbf{T}\mathbf{T}^*\mathbf{U}^* = \mathbf{U}\mathbf{T}^*\mathbf{T}\mathbf{U}^*$$

3. Simplify to $\mathbf{T}$:

Multiplying by $\mathbf{U}^*$ on the left and $\mathbf{U}$ on the right (since they are unitary) gives the condition on the triangular matrix $\mathbf{T}$:

$$\mathbf{T}\mathbf{T}^* = \mathbf{T}^*\mathbf{T}$$

The Schur form $\mathbf{T}$ must also be a normal matrix.

4. Proof that $\mathbf{T}$ must be Diagonal:

Let's look at the $(1, 1)$ element of both sides of $\mathbf{T}\mathbf{T}^* = \mathbf{T}^*\mathbf{T}$:

$$(\mathbf{T}\mathbf{T}^*)_{11} = \sum_{j=1}^n |\mathbf{t}_{1j}|^2 = |\mathbf{t}_{11}|^2 + |\mathbf{t}_{12}|^2 + \cdots + |\mathbf{t}_{1n}|^2$$

$$(\mathbf{T}^*\mathbf{T})_{11} = \sum_{j=1}^n |\mathbf{t}_{j1}|^2 = |\mathbf{t}_{11}|^2 + |\mathbf{t}_{21}|^2 + \cdots + |\mathbf{t}_{n1}|^2$$

Since $\mathbf{T}$ is upper triangular, $\mathbf{t}_{21} = \cdots = \mathbf{t}_{n1} = 0$. The equation becomes:

$$|\mathbf{t}_{11}|^2 + |$$